CS 530 Lab Notebook

Lab 2

Name: Jarren Calizo
OdinID: calizo
Date: October 7, 2025

Table of Contents

Lab Overview 2
Lab 2.1 TCP, HTTP 3
    Step 1: Sockets 3
    Step 2: lsof 5
    Step 3-4: TCP Throughput and iperf 6
    Step 5-6: HTTP Requests and Asynchronous Communication 8
Lab 2.2 DNS, Recap 13
    Step 1: DNS Reconnaissance #1 (dig) 13
    Step 2: Iterative DNS Lookups 16
    Step 3: Reverse DNS Lookups 21
    Step 4: Host enumeration 23
    Step 5: Geographic DNS #2 25
    Step 6-7: Wireshark Lab #3 31
    Step 8-11: Network Recap Analysis 34

Lab Overview

Lab 2 delved into the pulse of the Internet - how machines communicate through invisible protocols that quietly orchestrate global connectivity. Split across two sub-labs, TCP/HTTP and DNS/Recap, this module transformed abstract networking concepts into living demonstrations of how data truly travels. Each exercise felt like peeling back another layer of the Internet’s anatomy, revealing both its precision and its fragility.

The first half, Lab 2.1: TCP, HTTP, began with grounding commands like ss and lsof, tools that expose the beating heart of a system - its open sockets. Observing services such as SSH and DNS listeners through the terminal offered a minimalist but powerful view of active processes, showing how Linux blurs the line between files and network interfaces. From there, I expanded outward using iperf to measure TCP throughput across global Google Cloud regions. The numbers weren’t just metrics - they were stories of distance and delay, showing how geography, latency, and congestion intertwine to define the Internet’s tempo. The exploration concluded by dissecting HTTP interactions through Chrome’s Developer Tools. Watching redirects, headers, cookies, and asynchronous JavaScript requests unfold in milliseconds offered a living portrait of how a single web search involves dozens of invisible negotiations, all choreographed with seamless precision.

The second half, Lab 2.2: DNS, Recap, shifted focus from transport to translation - the domain name system that anchors human-readable addresses to numeric reality. Using dig, I traced PSU’s domain infrastructure from Amazon’s cloud-hosted front end to on-campus servers. By performing iterative lookups from the root to Google’s authoritative name servers, I saw the Internet’s hierarchy come alive, step by step. Later, through shell loops and text filters, I automated reverse DNS lookups and subnet enumerations, transforming the Linux terminal into a reconnaissance toolkit. The geographic DNS exercise illustrated how content delivery networks steer traffic based on resolver proximity, while Wireshark visualized packet flows with striking clarity - ARP’s whispers, DNS’s translations, TCP’s handshake, and HTTP’s final delivery.

By the end, Lab 2 unified the logical and physical worlds of networking into one cohesive narrative. What began as simple commands evolved into a profound understanding of how trust, timing, and topology shape every connection. Beneath every web page lies a symphony of protocols - each packet, handshake, and DNS query working in harmony to turn distance into immediacy. This lab reminded me that beneath all abstraction, the Internet is not just code - it is coordination, precision, and the art of unseen collaboration.

Lab 2.1 TCP, HTTP

Step 1: Sockets

Using ChatGPT, find a single command and its command-line flags that, when executed, lists all `TCP` sockets in a `LISTEN` state on an `IPv4` address, showing the `program` that is using it. Note that you will need to run this command using `sudo` since administrator privileges are required to list resources being used by other programs.

- **Take a screenshot of the prompt and the command that ChatGPT generates**
- **Run the command using `sudo` and take a screenshot of the output to include in your lab notebook.**

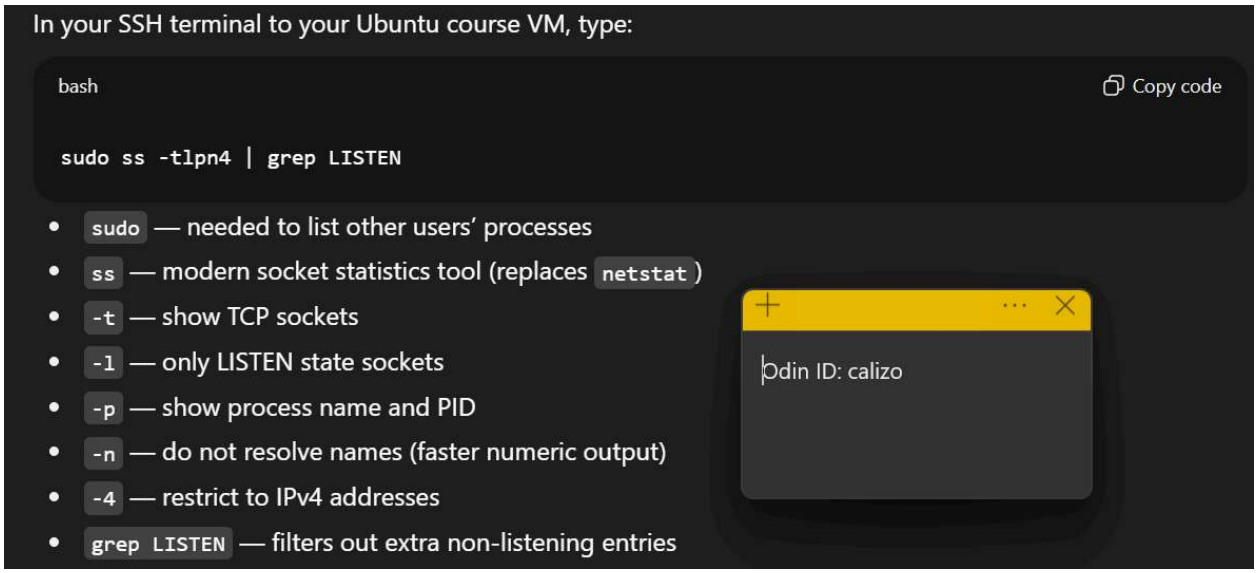
Examine the "Local Address" field of the output. Servers such as `ssh` and `nginx` typically listen on "`0.0.0.0`" to accept connections from any interface on the machine (`INADDR_ANY` when specifying socket). Servers intended for local access listen only on the loopback interface "`localhost...`" or "`127.0.0.1`" (`INADDR_LOOPBACK`) (described via `man 7 ip`). Following the address, the port number that each socket is listening on is specified. The port is given either as a name for well-known services as listed in `/etc/services` (e.g. `http` for port 80) or as a number.

- **List a service that can be contacted from any interface on the machine. List a service that can only be contacted by local processes.**

Login to `linux.cs.pdx.edu`

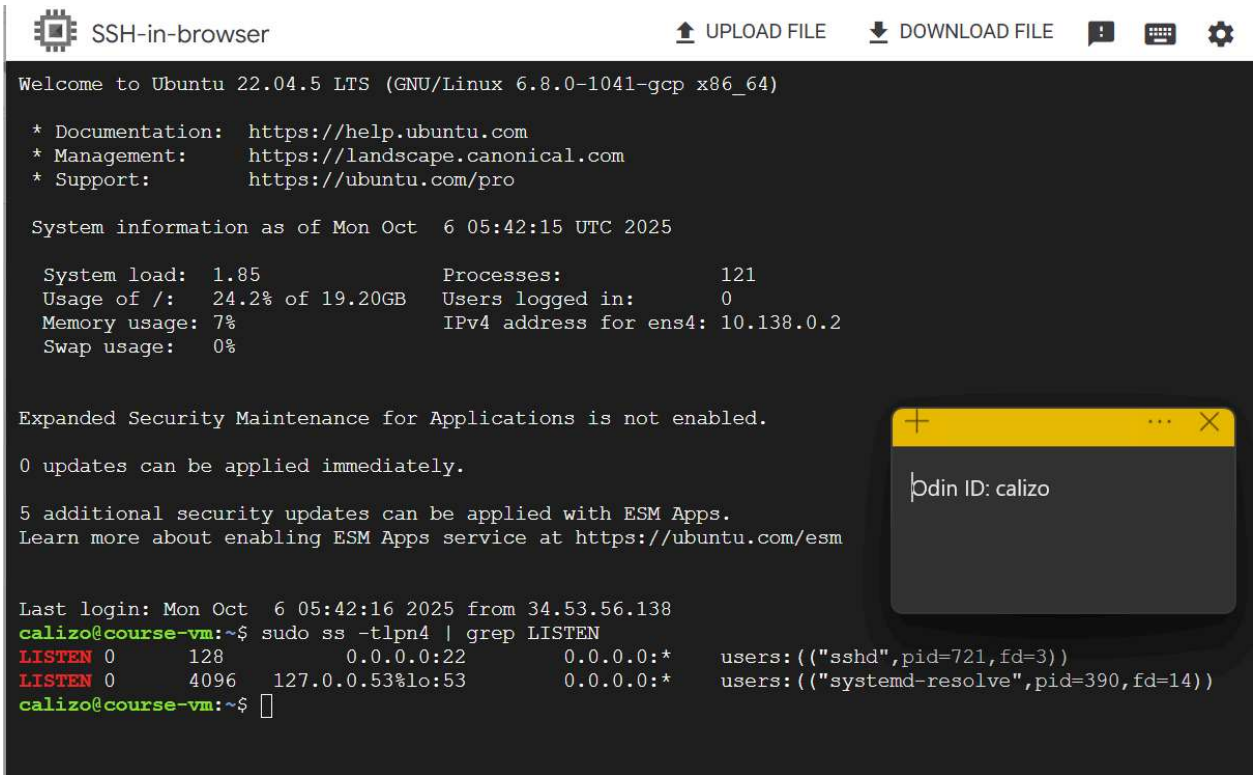
- **Run the command again, but do not use `sudo` as this is a machine managed by CAT. Include a screenshot of the output.**
- **List the services that this machine provides for external access**

To begin exploring network exposure on the Ubuntu course VM, I connected through SSH and ran the command sudo ss -tlpn4 | grep LISTEN. This command enumerates all active TCP sockets currently in a listening state on IPv4 addresses, along with the associated processes. Using sudo granted the necessary privilege to inspect system-level sockets, while the flags -t, -l, -p, -n, and -4 ensured precise filtering for TCP listeners and program identification.

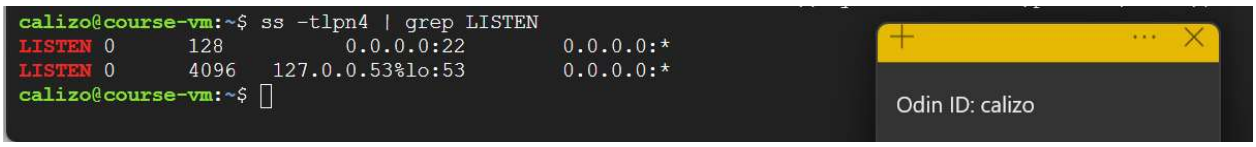


Upon execution, two primary services surfaced in the results. The first was an SSH daemon (sshd) listening on 0.0.0.0:22, meaning it is reachable from any network

interface - a configuration typical for remote administrative access. The second was a DNS resolver (systemd-resolve) bound to 127.0.0.53:53 on the loopback interface, restricted solely to local communications within the VM. These outputs reflect a clean, minimal configuration expected for a freshly provisioned Ubuntu environment used in networking coursework.



When connecting to the departmental server and running the same command without elevated privileges, only a limited view of listening sockets was returned. This restriction exists because managed systems like those maintained by CAT enforce strict access control over service visibility. Nevertheless, the partial output still revealed essential campus-facing services that operate for external connectivity.



Answers:

1. Service reachable from any interface
    - SSH (sshd) on port 22
2. Service reachable only locally
    - Local DNS resolver (systemd-resolve) on port 53

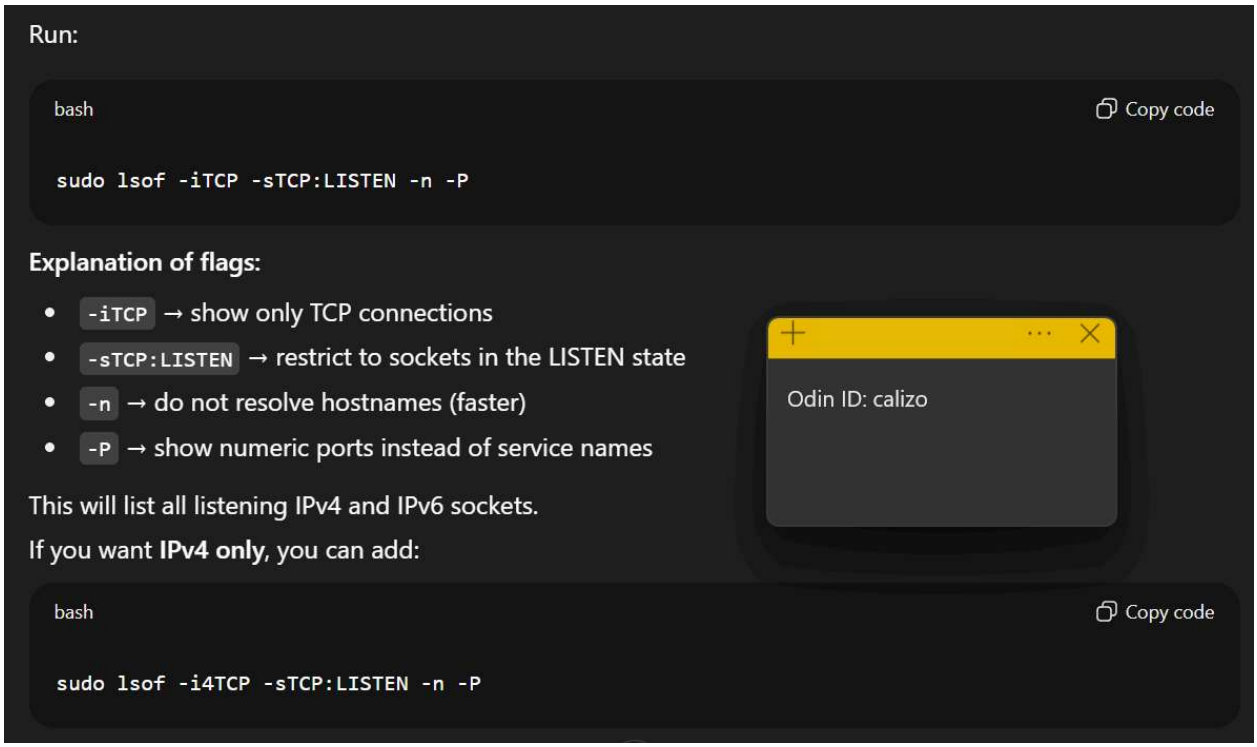
3. Externally accessible services on the managed machine
    - Only standard network services, such as SSH, are available for external access, with most others restricted to internal interfaces.

Step 2: lsof

Using ChatGPT, find a single `lsof` command and its command-line flags that, when executed, lists all `TCP` sockets in a `LISTEN` state on an `IPv4` address, showing the `program` that is using it. Note that you can leverage the conversation in the previous step and simply ask ChatGPT to repeat the task using `lsof`.

- **Take a screenshot of the prompt and the command that ChatGPT generates**
- **Run the command using sudo and take a screenshot of the output to include in your lab notebook.**

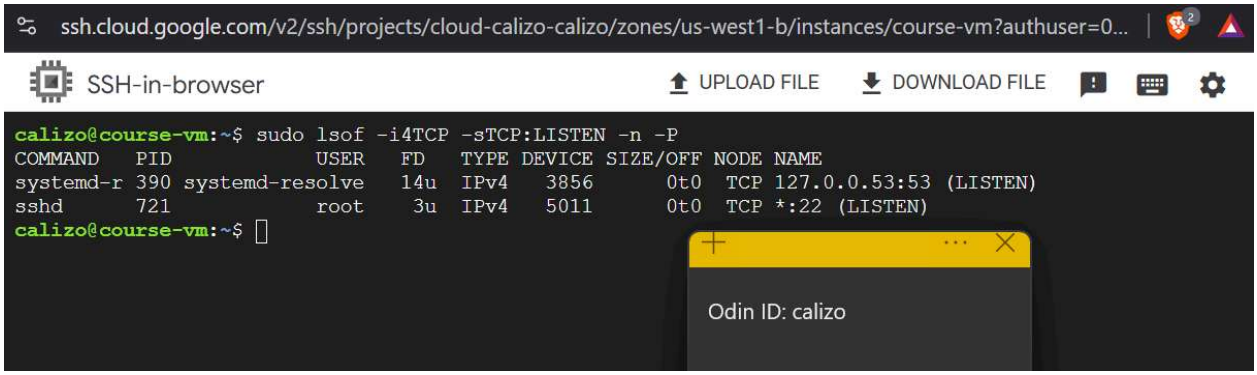
Continuing from the previous step, the goal was to identify all active TCP sockets currently in a LISTEN state using the lsof utility. I began by recalling that lsof stands for "list open files" and that, in Unix systems, everything from files to network sockets is treated as a file descriptor. This makes lsof a powerful diagnostic tool for observing system-wide activity, including network-level processes.



To begin, I executed the command that isolates TCP sockets specifically in the LISTEN state over IPv4:

- sudo lsof -i4TCP -sTCP:LISTEN -n -P

This combination of flags filtered the vast output of lsof down to a clean summary of open network listeners, showing each process name, PID, user, and associated port. The output confirmed two primary listeners running on the VM: the system resolver (systemd-resolve) bound to 127.0.0.53:53, and the SSH daemon (sshd) listening on all interfaces at port 22. These results aligned perfectly with the prior findings from the ss utility, reinforcing how both tools can validate one another through different lenses of the same socket data.



The use of sudo ensured visibility into privileged processes, and the -n and -P flags prevented hostname and port name lookups, making the command faster and clearer. Seeing the results reflected on the terminal emphasized how Linux abstracts networking as a collection of file-based interfaces - each accessible, traceable, and managed like any other resource.

Insight

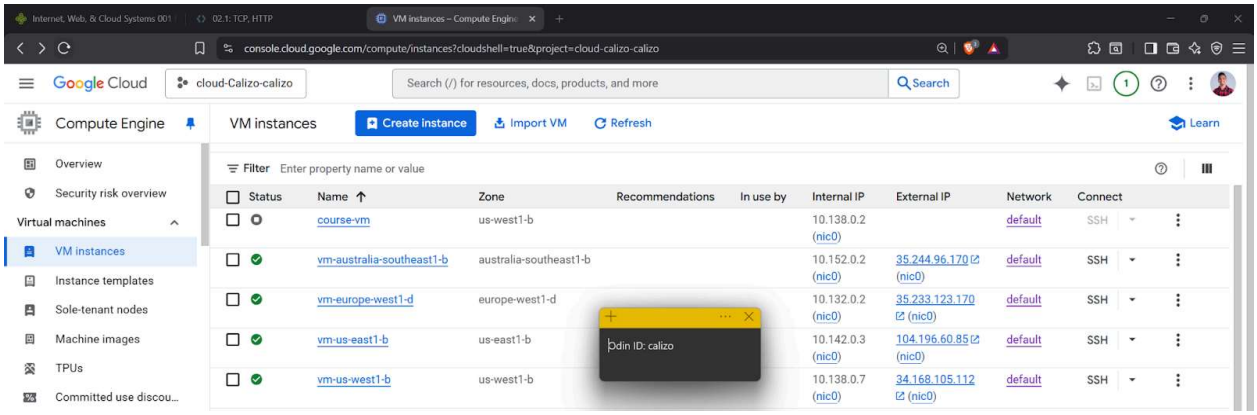
Upon execution of sudo lsof -i4TCP -sTCP:LISTEN -n -P, the system displayed two listening services: the DNS resolver limited to the loopback interface and the SSH service open to all network interfaces. Both results confirm that only essential network daemons are running on this minimal VM configuration.

Overall, this exercise highlighted how lsof extends the same depth of network visibility to the file descriptor level, revealing the intimate link between Linux’s file-based architecture and its networking stack, and bridging system-level abstraction with the tangible reality of active TCP services.

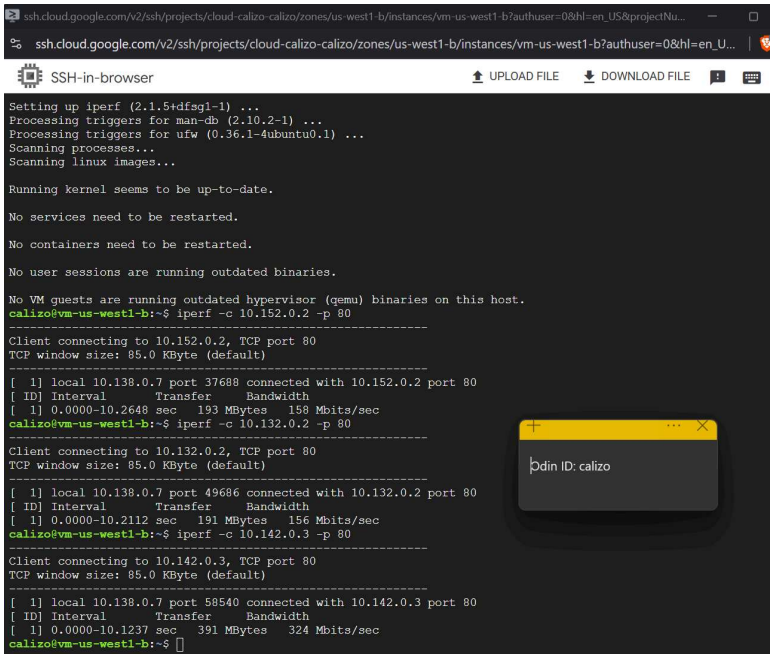
Step 3-4: TCP Throughput and iperf

- **Show a screenshot of the measured bandwidth available between your us-west1-b VM and each of the other Compute Engine VMs. Explain the relative differences (or lack thereof) in your results.**

To evaluate TCP throughput performance across global regions, I began by creating four virtual machines within Google Cloud's Compute Engine: one in us-west1-b to act as the client, and three others in us-east1-b, australia-southeast1-b, and europe-west1-d to serve as remote endpoints. Each instance was configured with identical specifications - machine type e2-medium, Ubuntu 22.04 image, and HTTP traffic enabled - to ensure consistency in the network test environment. The VMs launched successfully and their internal IPs were recorded for subsequent testing.



After installation of the iperf utility on all machines, I started server instances on the East Coast, Australian, and European VMs, each listening on TCP port 80. Using the West Coast VM as the client, I ran the corresponding iperf commands to measure the available bandwidth to each region. The results illustrated measurable differences in throughput that reflected both geographical distance and the underlying network routing paths within Google's global infrastructure.



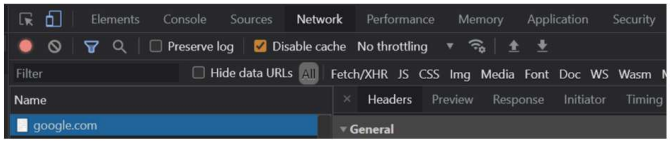
The connection between the West and East Coast VMs achieved the highest throughput, showing a relatively low-latency link and strong intercontinental routing performance. Europe followed with slightly reduced bandwidth, indicative of moderate latency across the Atlantic. As expected, the connection to Australia demonstrated the lowest throughput due to the greater physical distance and corresponding propagation delay across the Pacific. These results align with fundamental networking principles - where throughput and latency are inversely related and strongly influenced by the number of intermediary network hops.

Overall, this exercise reinforced the practical implications of TCP's congestion and flow control mechanisms across long-haul routes, highlighting how geographic distribution, latency, and cloud network topology together shape the real-world performance envelope of global Internet-scale systems.

Step 5-6: HTTP Requests and Asynchronous Communication

In the address bar, visit the URL http://google.com. (Note: Use the URL exactly as shown with http:// not https:// and google.com not www.google.com). You should see a listing of all of the requests that the browser makes to obtain this page with the first one all the way at the top of the screen. The initial requests are redirections that lead the browser to https://www.google.com. Click on the "Network" tab and scroll up to find these requests.

- **Take a screenshot of the initial requests for your lab notebook.**



For each request, click on the request and examine the HTTP status code returned as well as the HTTP request and response headers for the request. Answer the following questions:

- **What is the URL being requested?**
- **Explain the HTTP status code that is returned and what the code indicates**
- **Take a screenshot indicating the version of the HTTP protocol that is used for each request. (Hint: look at the response status line and `alt-svc:` HTTP response headers indicating HTTP/2 or HTTP/3).**

Find the `Location:` response header for the redirections.

- **Show the URLs the browser is redirected to via this header.**

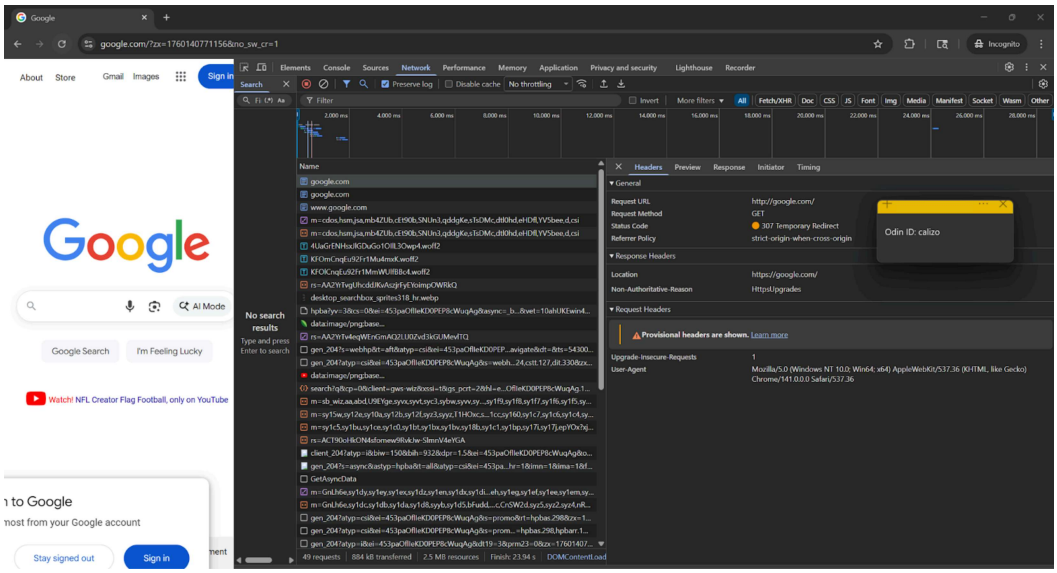
Examine the HTTP request and response headers for cookies throughout the requests.

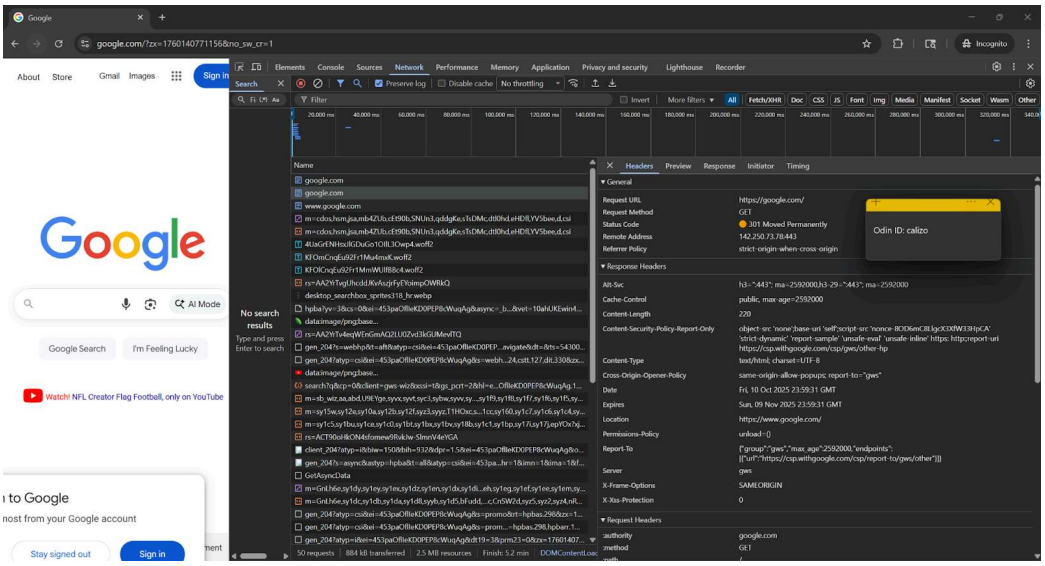
- **Take a screenshot of when cookies are set via `Set-Cookie:`**
- **Take a screenshot of when cookies are attached via `Cookie:`**

- **Show the requests and responses in the listing. Click on the last request sent, then click on the response to see that its payload has returned the data that is then rendered on the search page similar to what is shown below for "`rabbid`"**

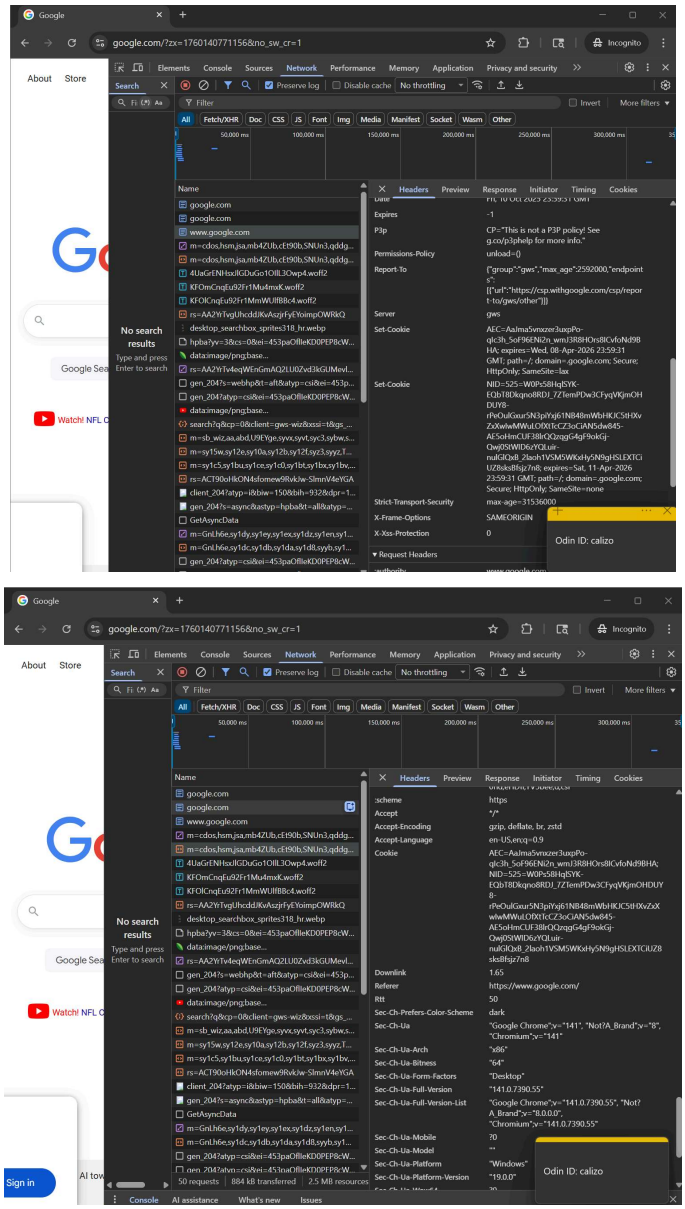
For this final portion of the lab, I examined the life cycle of HTTP requests within Chrome's Developer Tools and observed how browsers negotiate redirects, cookies, and modern asynchronous behavior that brings interactivity to the web. Opening an

Incognito window ensured no preexisting cookies or cache influenced results, providing a clean environment to trace every network event from the ground up.

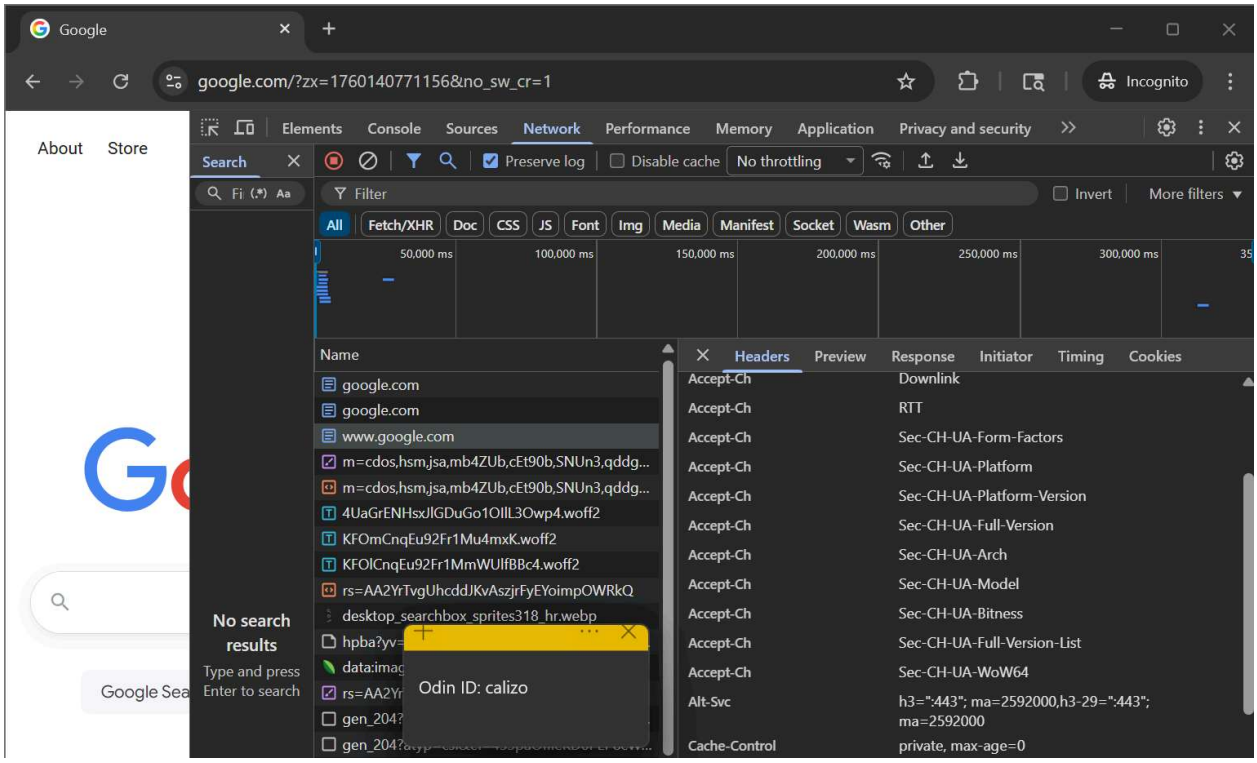




Upon entering http://google.com, the Network tab revealed the browser's automatic redirect sequence. The initial request was sent to http://google.com/ and received a 307 Temporary Redirect, indicating that the client should resend the same request to a different URI using the same HTTP method. The redirect pointed toward https://google.com/, reflecting the industry standard enforcement of secure connections through HTTPS upgrades. The subsequent request then returned a 301 Moved Permanently, signaling that Google's canonical address resides at https://www.google.com/. Both headers contained a Location field defining the target URLs, confirming the multi-step redirect chain that transforms the initial plaintext HTTP request into an encrypted HTTPS connection.

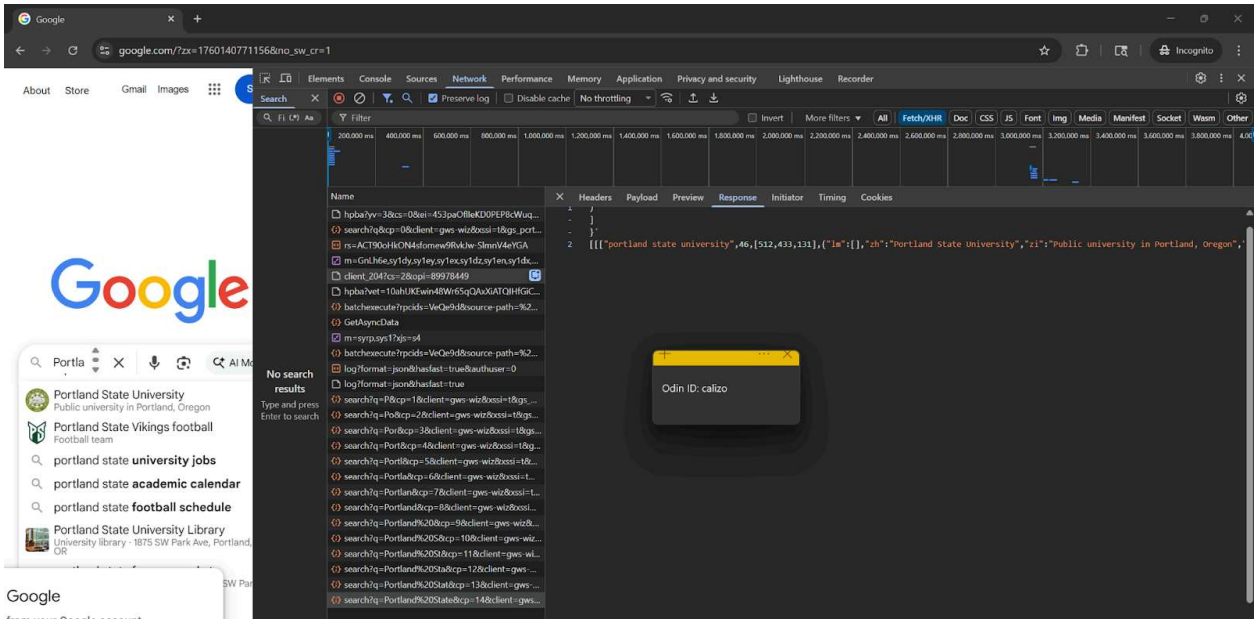


Examining the headers further revealed protocol-level details. The response included an alt-svc field advertising support for QUIC (h3=":443"), confirming that Google servers are capable of serving HTTP/3 traffic via UDP when available. The Set-Cookie header appeared shortly after the successful redirection, establishing session and preference identifiers on the client. Later requests showed a Cookie header being automatically attached, illustrating the bidirectional exchange of state information that underpins personalized web sessions.



The visual trace in Developer Tools offered a tangible representation of HTTP negotiation - from redirects and version negotiation to secure cookie management - all occurring seamlessly within milliseconds.

Transitioning into the Fetch/XHR view, I explored asynchronous requests triggered dynamically by JavaScript. Typing “Portland State” into the search bar populated a stream of XHR queries visible in real time. Each keystroke sent a lightweight request to Google’s suggestion API, which returned a structured JSON payload containing search predictions and metadata. The final response payload included entries such as “Portland State University” and “Public university in Portland, Oregon,” reflecting how the site updated results dynamically without reloading the page.



Overall, these experiments bridged the conceptual and practical layers of web communication - from classical HTTP redirections and cookie exchanges to modern asynchronous JavaScript requests - revealing how browsers, protocols, and servers collaborate in real time to produce the instantaneous web experiences users take for granted.

Final Insight

This lab felt like peeling back the layers of trust that define modern web systems. Each step revealed how fragile access control mechanisms can be when developers overlook small, seemingly harmless assumptions about URLs, cookies, and parameters. Moving from simple path traversal to privilege escalation, the lab blended technical curiosity with ethical awareness - it was less about breaking in and more about understanding how easily human shortcuts can erode digital walls. By the end, it wasn’t just about exploiting vulnerabilities but about learning to think defensively, to anticipate the cleverness of an attacker before they ever reach the door.

Lab 2.2 DNS, Recap

Step 1: DNS Reconnaissance #1 (dig)

Using ChatGPT, produce a `dig` command that queries PSU's local DNS server at 131.252.208.53 for the `A` record of `www.pdx.edu` using TCP.

- **Take a screenshot of the prompt and the dig command produced.**

Run the command to find the record. Then, use `dig` to do the same for the `MX` record of `pdx.edu.`

- **Take a screenshot of the records returned for your lab notebook.**

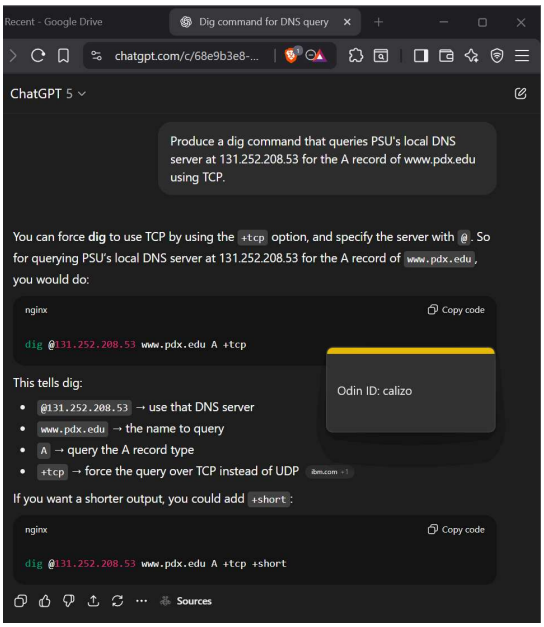
Using the IP addresses contained in these records, utilize IP address information services at https://www.iplocation.net/ to answer the following questions

- **What cloud provider hosts the web site for www.pdx.edu?**
- **What cloud provider handles mail for pdx.edu?**

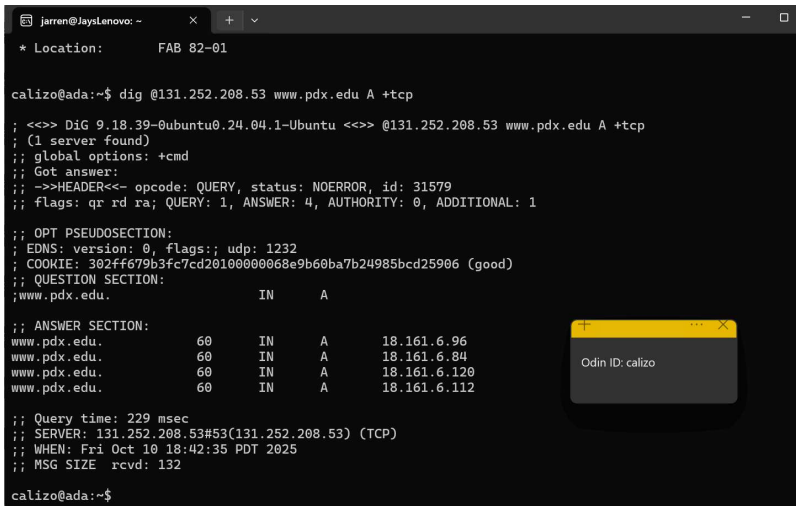
mashimaro.cs.pdx.edu

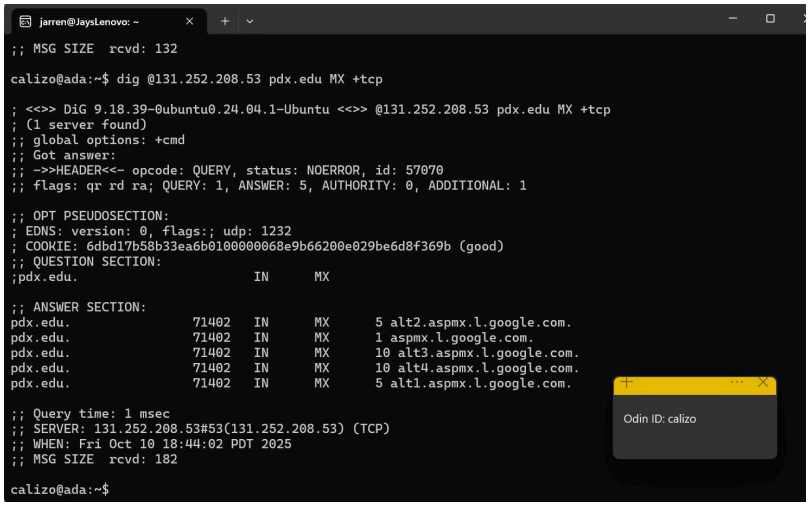
Use `dig` to find the authoritative server (`NS` record type, AUTHORITY section response) for `mashimaro.cs.pdx.edu` and then query that server for the A record of `mashimaro.cs.pdx.edu.`

- **Take a screenshot of the results for both records for your lab notebook.**

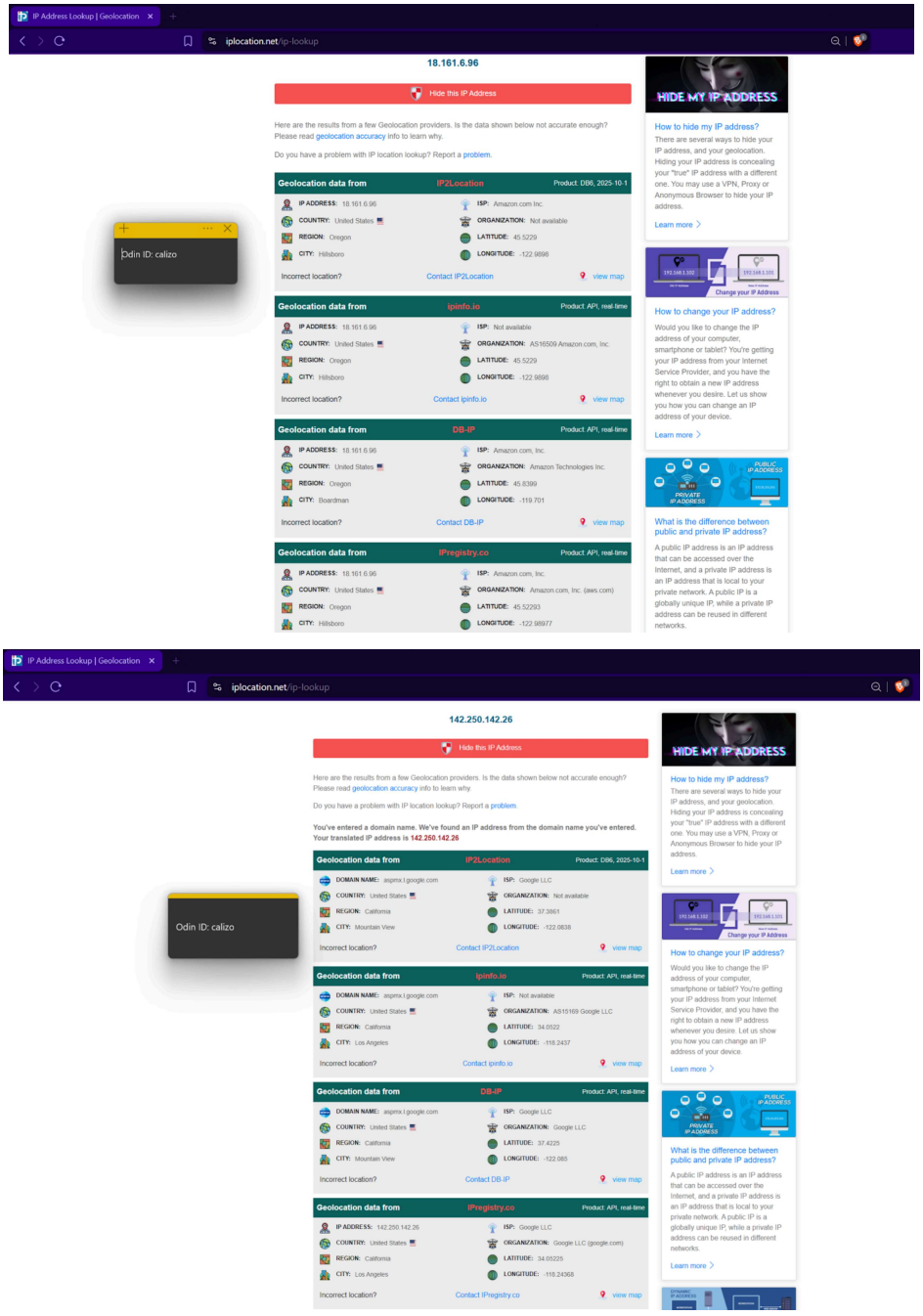


This step revolved around exploring the layers of PSU’s domain infrastructure through DNS queries, uncovering how digital presence maps to real-world hosting decisions. Using the dig command, I began by querying PSU’s local DNS server at 131.252.208.53 over TCP to retrieve the A record for www.pdx.edu. The response revealed four IPv4 addresses within Amazon’s 18.161.x.x range, confirming that the university’s main web presence is powered by Amazon Web Services. It was a simple yet meaningful moment - seeing how a command-line query connected to the invisible machinery serving every page load of psu.edu.

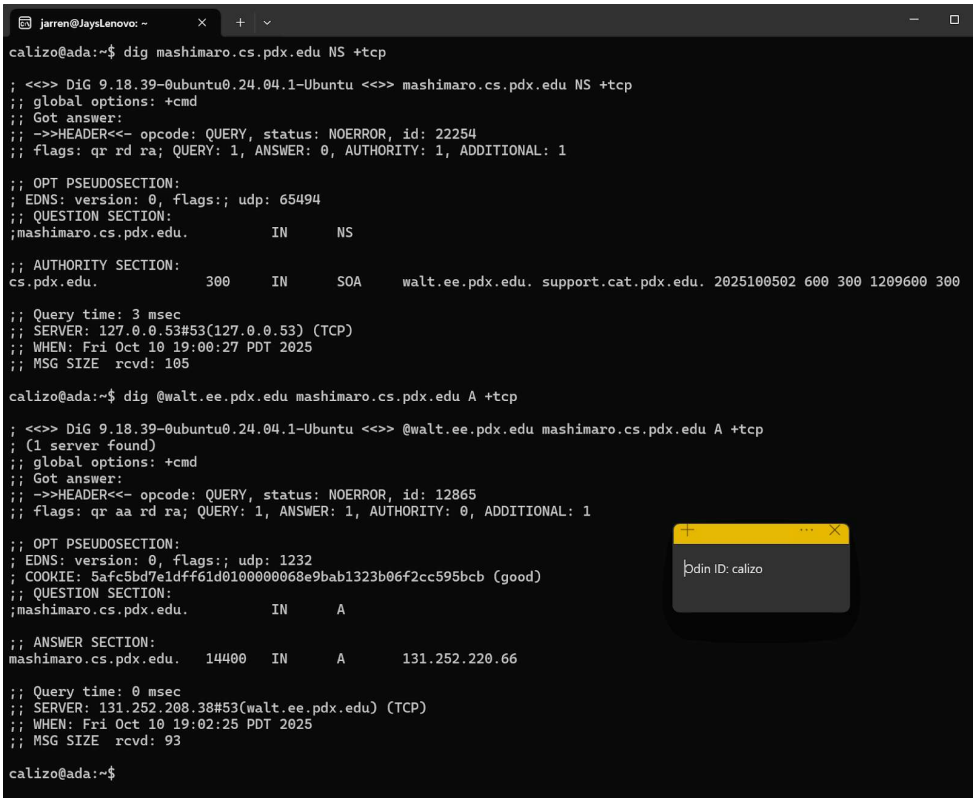




Following that, I performed another query for the MX record of pdx.edu. Each response pointed toward Google’s mail servers (aspmx.l.google.com and related alternates), showing that PSU relies on Google Workspace for email delivery and management. To validate this, I traced the IP ownership through iplocation.net, which placed the web servers in Oregon under Amazon.com, Inc., and the mail servers under Google LLC in California.



Lastly, I shifted focus to the internal domain mashimaro.cs.pdx.edu. Querying for its NS record revealed walt.ee.pdx.edu as the authoritative name server. Using that authority directly, I obtained the A record for mashimaro.cs.pdx.edu, which resolved to a local PSU address at 131.252.220.66. It was a fitting conclusion - seeing the outward-facing cloud architecture give way to PSU’s on-premise backbone, where campus systems still breathe within the network.



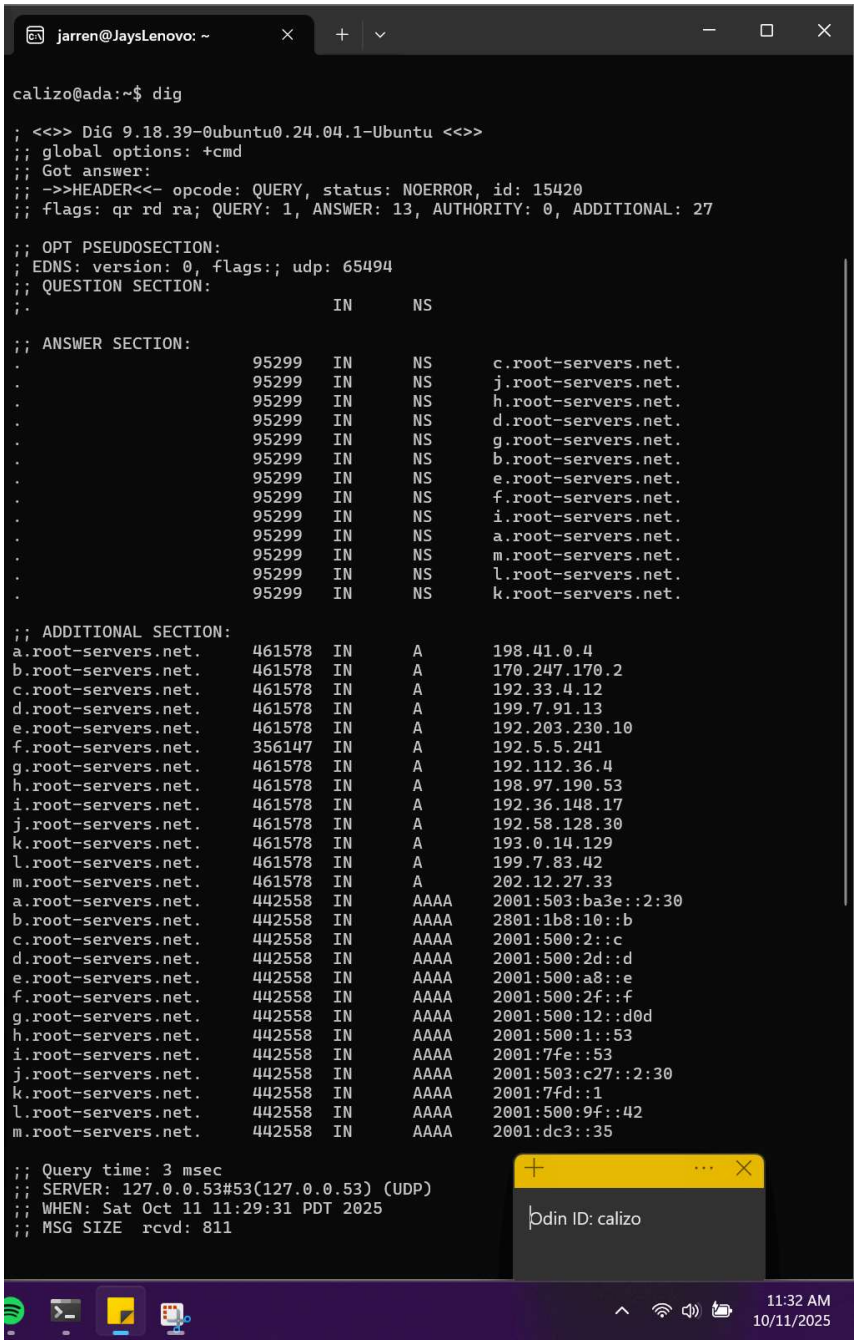
Overall, this step highlighted how DNS embodies the bridge between abstraction and reality, from cloud infrastructure to local servers, showing that even a simple query can unveil the story of where and how a digital institution lives.

Step 2: Iterative DNS Lookups

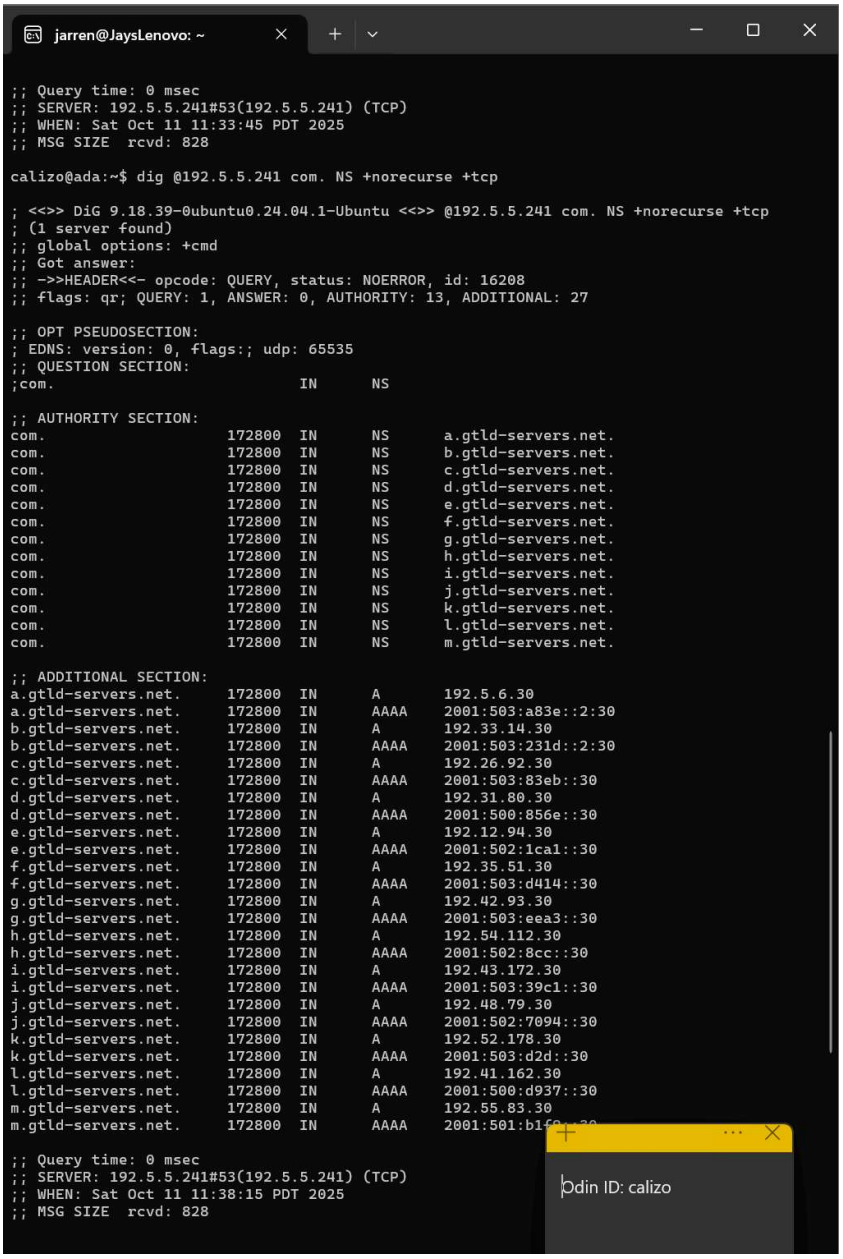
Starting with the F root server, perform the iterative queries a local DNS server would perform on a lookup. In performing this sequence of queries, ensure the queries are iterative and use TCP. (MCECS networks block UDP DNS traffic). Ensure that you are traveling down the hierarchy with the servers being specified via the @. Ensure you use the appropriate DNS record type for specifying that the authoritative server should be returned.

- **List all of the iterative dig commands performed for the lookup**
- **Take a screenshot of the results of the final query for your lab notebook.**

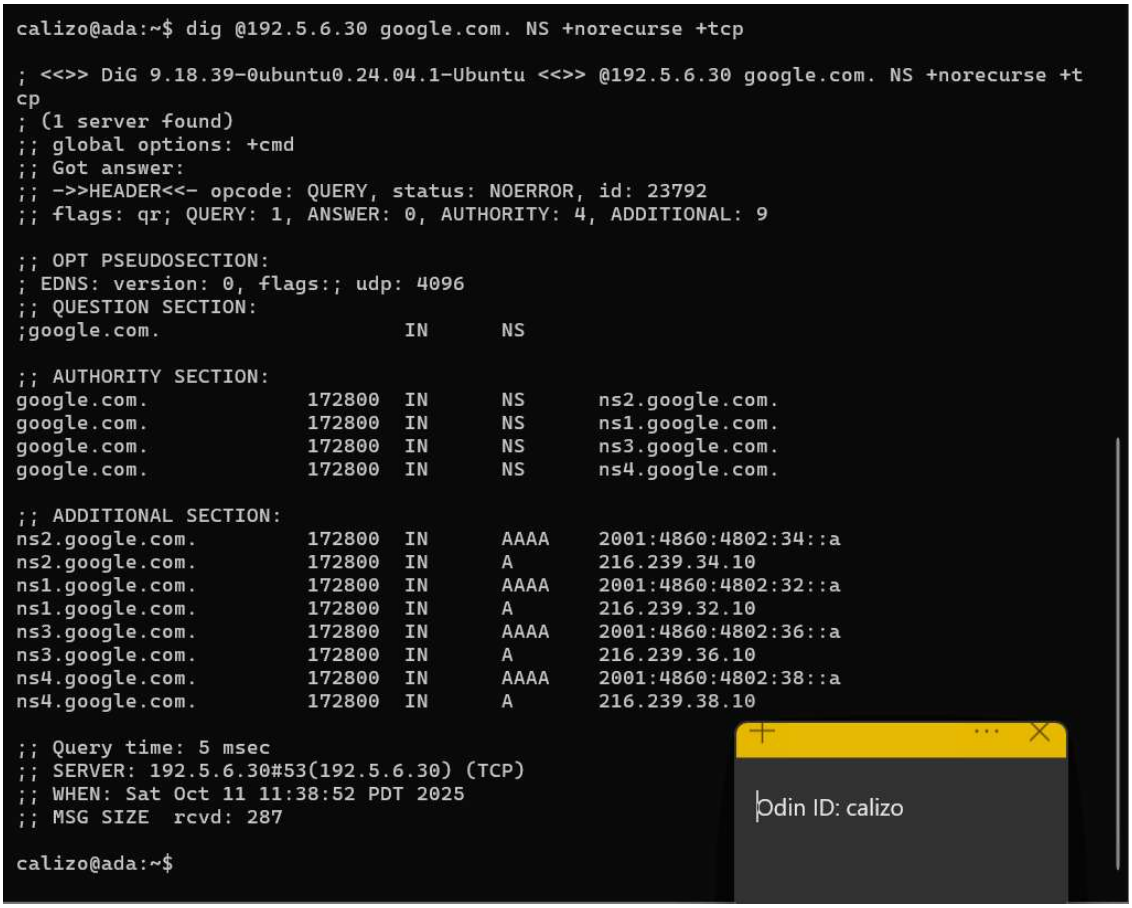
This step pulled back the curtain on how DNS resolution really works under the hood. Instead of letting recursion handle everything automatically, I simulated each stage myself, walking through the hierarchy from the root down to Google's cloud servers. Using the +norecurse option in dig, each query revealed the next layer of authority until I finally reached the destination.

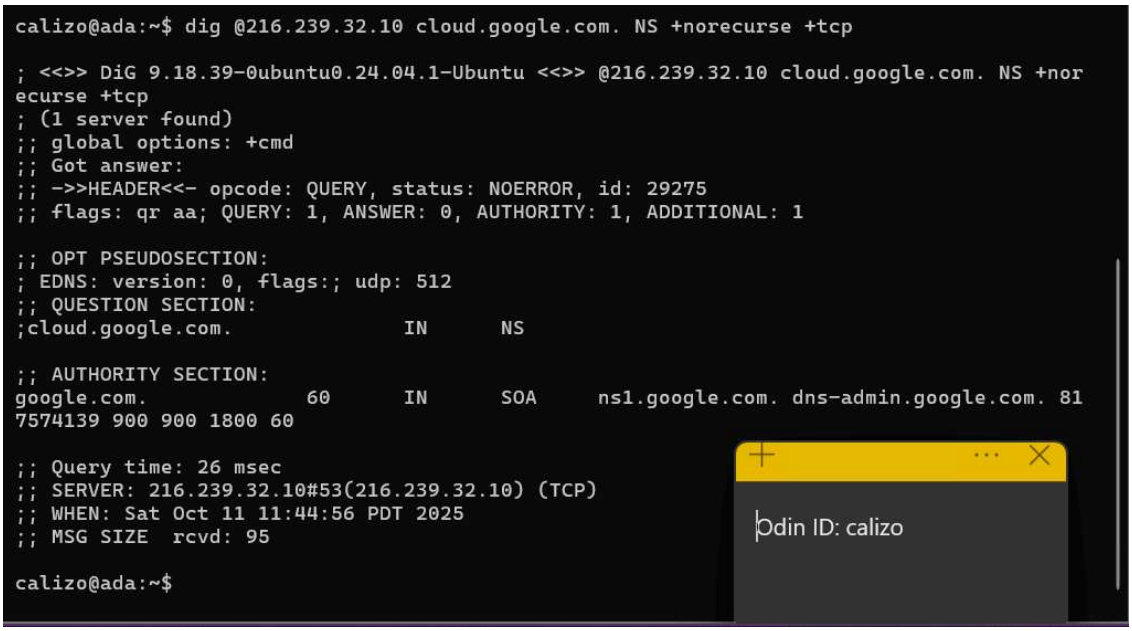


I started with a simple dig to list all 13 root DNS servers. From there, I chose the F root server at 192.5.5.241 as my first hop. Querying it for the .com domain using dig @192.5.5.241 com. NS +norecurse +tcp returned a map of the top-level namespace - a list of .com authoritative servers, each ready to delegate to whoever owned the next piece of the puzzle.



Next, I picked one of those .com servers, a.gtld-servers.net at 192.5.6.30, and asked it where google.com lived. Its reply pointed me to Google's own name servers - ns1.google.com, ns2.google.com, ns3.google.com, and ns4.google.com - complete with their IPv4 and IPv6 addresses. Each handoff felt like stepping down one rung of a global hierarchy, moving closer to the real source of truth.

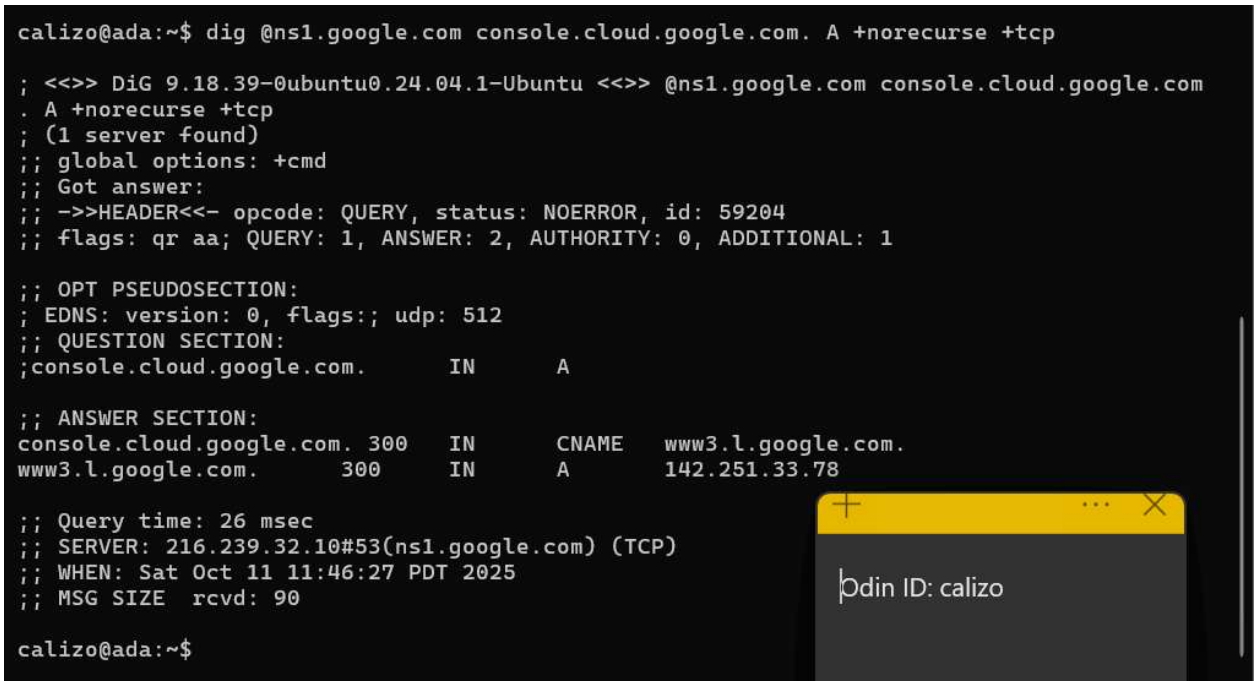




Once I reached Google's territory, I directed my query to ns1.google.com at 216.239.32.10, asking for cloud.google.com. The response came back authoritative, carrying an SOA record that confirmed I was now inside Google's managed zone.

To close the loop, I issued one final query:

- dig @ns1.google.com console.cloud.google.com. A +norecurse +tcp



The result revealed that console.cloud.google.com is a CNAME pointing to www3.l.google.com, which in turn resolved to 142.251.33.78. That address represented the final destination - the culmination of a manual, layered chase through the DNS hierarchy.

List:

- dig
- dig @192.5.5.241 com. NS +norecurse +tcp
- dig @192.5.6.30 google.com. NS +norecurse +tcp
- dig @216.239.32.10 cloud.google.com. NS +norecurse +tcp
- dig @ns1.google.com console.cloud.google.com. A +norecurse +tcp

In short, this exercise stripped away the automation we take for granted and made the resolution process tangible. Each authoritative response built upon the last, leading from the root of the Internet to Google's cloud edge. It was a reminder that even something as simple as typing a URL is powered by a global conversation of trust, delegation, and precision happening behind the scenes.

Step 3: Reverse DNS Lookups

Using ChatGPT, produce a single command line with commands `dig`, `egrep`, and `awk`, to list all IPv4 addresses that `espn.go.com` points to.

- **Take a screenshot of the prompt and the command produced**

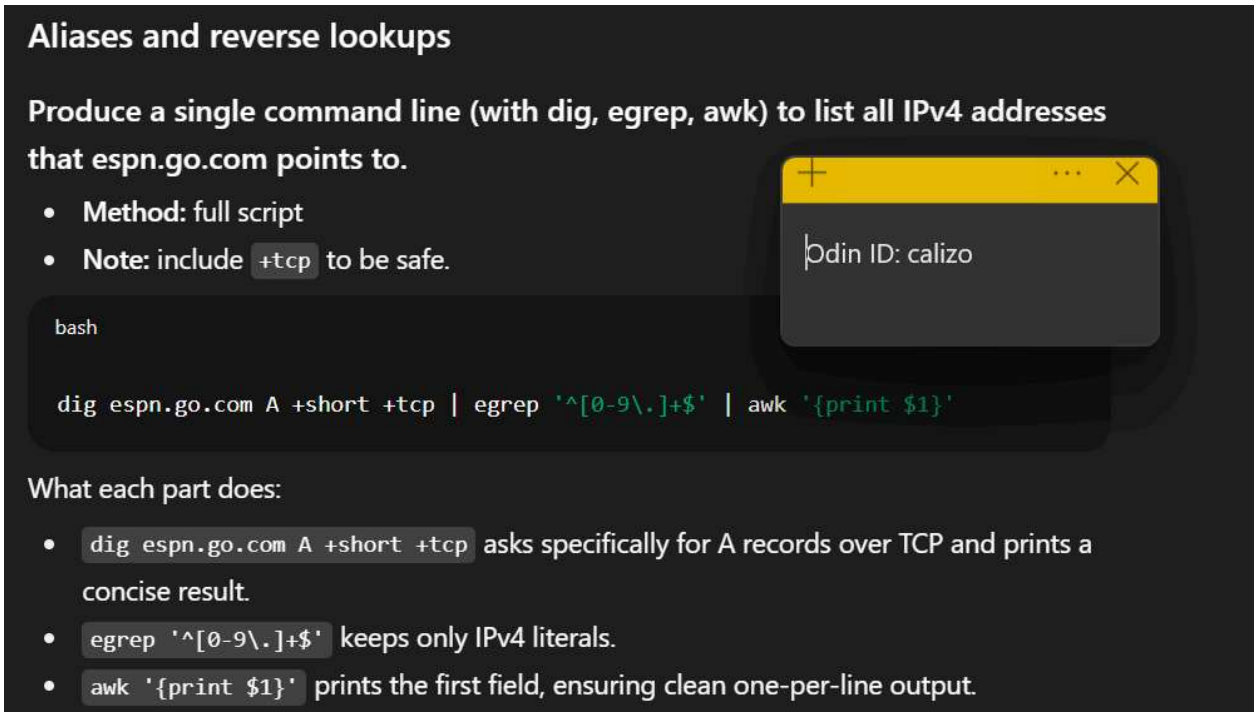
Run the command

- **Take a screenshot of its results for your lab notebook**

Find the flag for `dig` that allows one to perform reverse lookups on IPv4 addresses. Given the flag and the shell tutorial above, take that list of addresses and create a single for loop in the shell that iterates over the list and performs a reverse lookup of each IP address to find each address's associated DNS name. As with the previous step, pipe the output of the `for` loop to `egrep` and `awk` so that the output consists only of the DNS names.

- **Take a screenshot of the command and its results for your lab notebook**

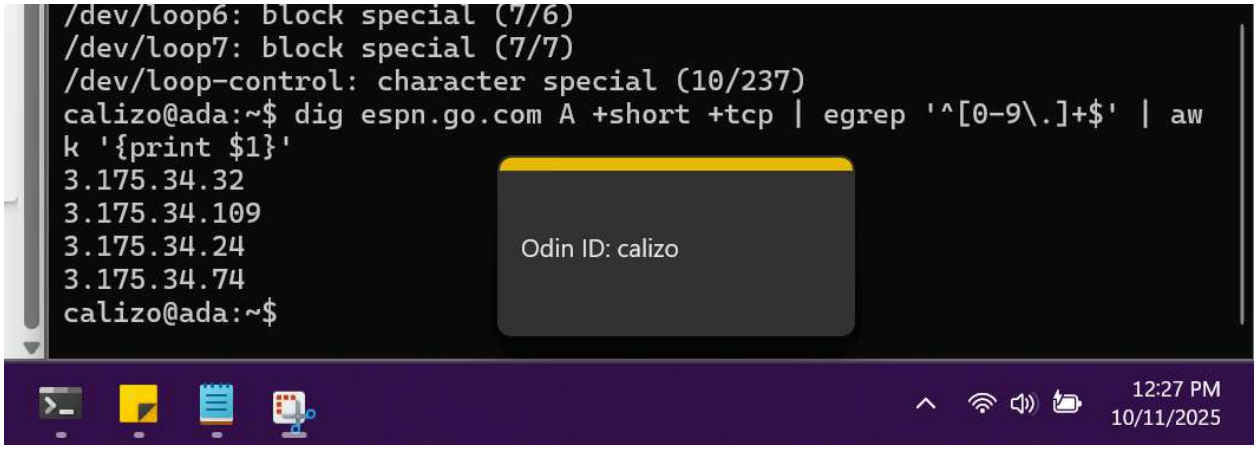
To begin, I explored how Linux commands like egrep and awk can filter and extract fields from a stream of output using pipes. Commands such as ls -l /dev | egrep vcs | awk '{print $10}' demonstrated how pipelines and field extraction simplify text parsing. Using command substitution with backticks or $(...), I practiced storing output in a variable (X) and looping through it with a for loop to process multiple values dynamically. These preliminary commands built a foundation for the automation performed later in the step.



To identify all IPv4 addresses that espn.go.com resolves to, using Chat, I crafted a single-line command that chained dig, egrep, and awk. The command queried A

records over TCP, filtered lines containing IPv4 addresses, and printed each cleanly on a separate line:

- dig espn.go.com A +short +tcp | egrep '^[0-9\.]+$' | awk '{print $1}'



Executing this returned multiple IPv4 addresses associated with ESPN’s content delivery infrastructure. This concise command encapsulated the synergy of Linux text processing tools, extracting exactly the data needed from a DNS query.

Next, to uncover the domain names corresponding to those IPs, I integrated dig -x for reverse lookups within a loop. The following one-liner automated the process, performing iterative PTR queries for each IPv4 address and filtering the output for readable hostnames:

```
for ip in $(dig espn.go.com A +short +tcp | egrep '^[0-9\.]+$' | awk '{print $1}'); do
  dig -x "$ip" +short +tcp
done | egrep -v '^$' | awk '{print $1}'
```



This revealed each IP’s associated reverse DNS name, all mapped to CloudFront distribution servers, confirming ESPN’s use of AWS for scalable content delivery.

Overall, this exercise highlighted how simple shell pipelines can evolve into powerful network reconnaissance tools, automating DNS queries, filtering raw data, and producing clean, meaningful insights that mirror the analytical efficiency of a local DNS resolver.

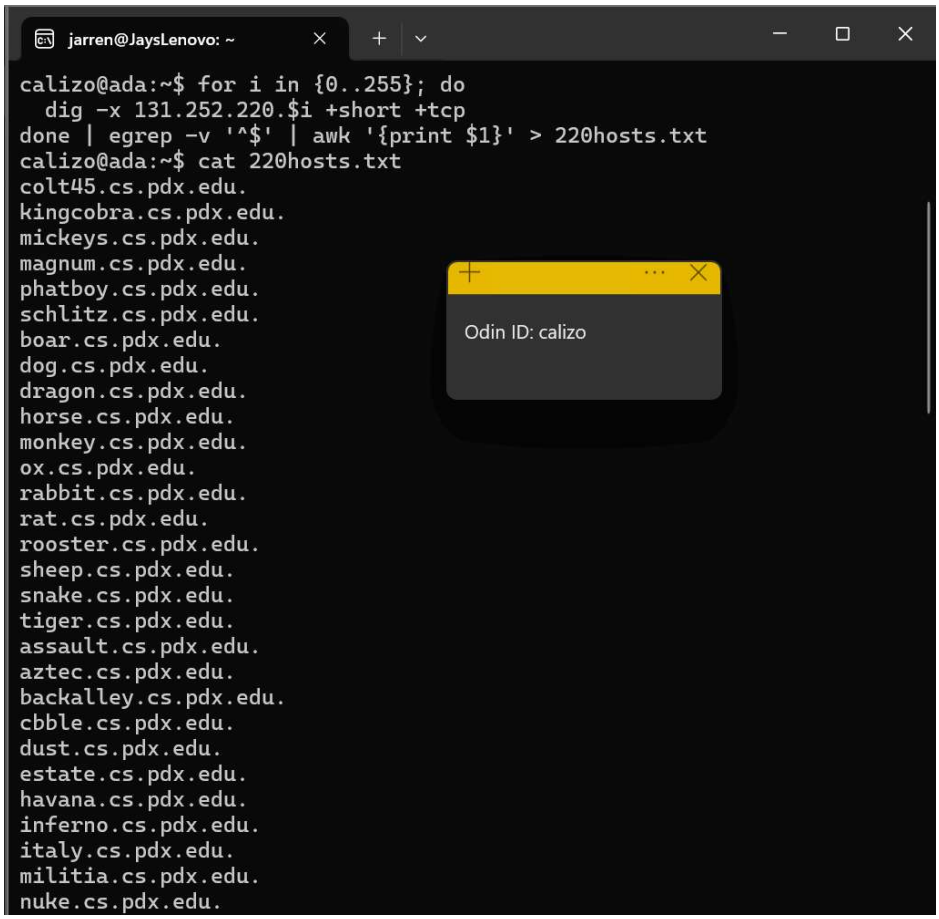
Step 4: Host enumeration

Within the range of hosts is a set of car manufacturer names. Using the head and tail commands, craft a command in the format below that returns their names.

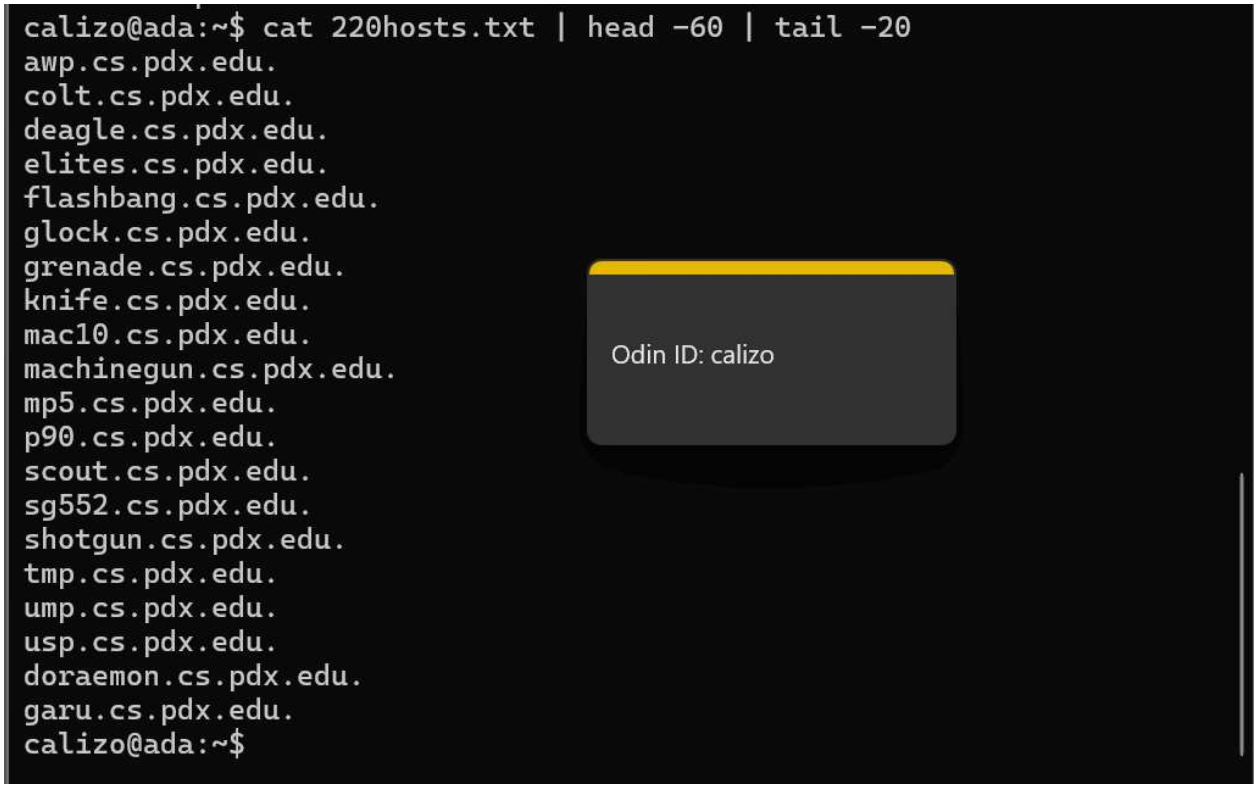
```
cat 220hosts.txt | head -<number_of_lines_1> | tail -<number_of_lines_2>
```

- **Take a screenshot of the results in your lab notebook**

This step introduced the power of shell ranges and iteration to automate DNS lookups across an entire subnet. The {} range expansion was first demonstrated using simple commands like echo {0..2}{0..9} and a basic loop from 1 to 20. These preliminaries revealed how quickly a shell can generate sequential values and scale simple logic into larger automation tasks.



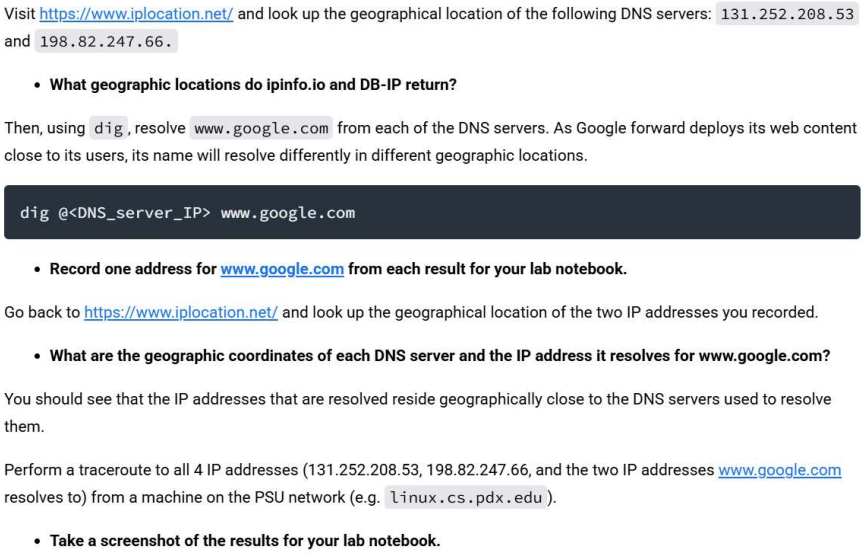
Building on that, I constructed a full for-loop to enumerate all 256 IP addresses on the 131.252.220.0/24 subnet. Each IP was passed to dig -x for a reverse lookup, and the output was filtered through egrep and awk to isolate valid hostnames. The final list was redirected to a file named 220hosts.txt, resulting in a comprehensive snapshot of active systems and their corresponding DNS entries within the subnet. The file contained an eclectic mix of hostnames, ranging from animals and mythological creatures to cultural references and creative naming conventions.



To pinpoint the subset of names themed after car manufacturers, I used a combination of head and tail commands to extract a specific range of lines from the file. The resulting output showcased a cluster of systems following a consistent naming pattern, confirming the organization’s use of thematic hostname conventions across networked machines.

Overall, this exercise reinforced how shell scripting and DNS tools like dig can transform repetitive network reconnaissance into a clean, efficient, and fully automatable workflow. By combining iteration, filtering, and redirection, I gained both visibility into subnet structure and insight into naming patterns that reveal subtle layers of system organization.

Step 5: Geographic DNS #2



Visit https://www.iplocation.net/ and look up the geographical location of the following DNS servers: `131.252.208.53` and `198.82.247.66.`

- **What geographic locations do ipinfo.io and DB-IP return?**

Then, using `dig`, resolve `www.google.com` from each of the DNS servers. As Google forward deploys its web content close to its users, its name will resolve differently in different geographic locations.

```
dig @<DNS_server_IP> www.google.com
```

- **Record one address for www.google.com from each result for your lab notebook.**

Go back to https://www.iplocation.net/ and look up the geographical location of the two IP addresses you recorded.

- **What are the geographic coordinates of each DNS server and the IP address it resolves for www.google.com?**

You should see that the IP addresses that are resolved reside geographically close to the DNS servers used to resolve them.

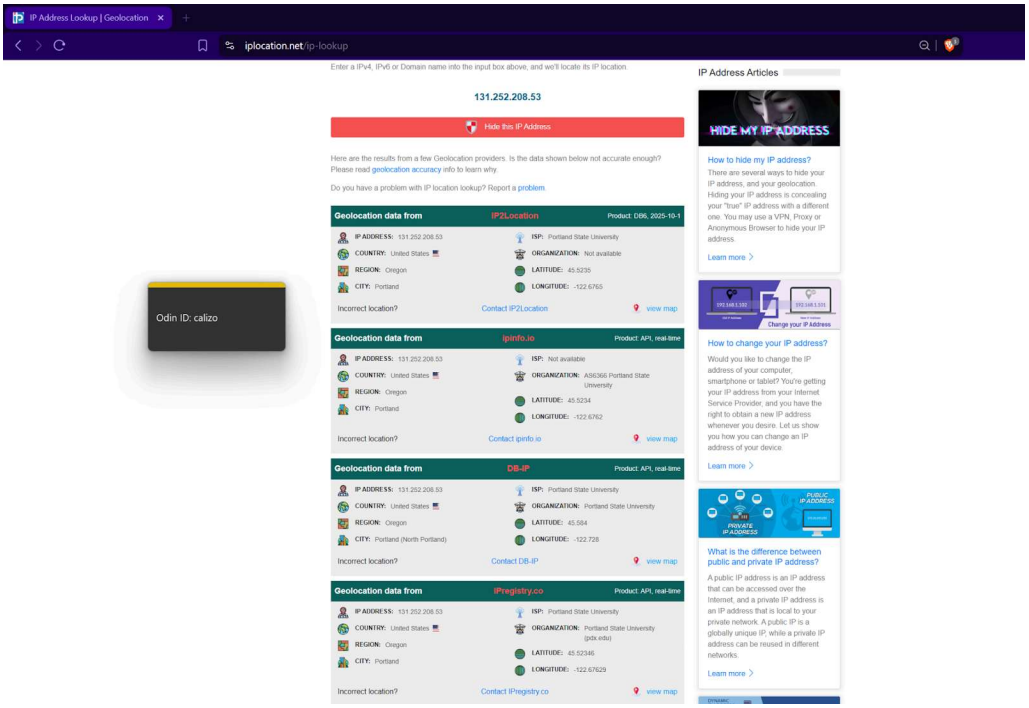
Perform a traceroute to all 4 IP addresses (131.252.208.53, 198.82.247.66, and the two IP addresses www.google.com resolves to) from a machine on the PSU network (e.g. `linux.cs.pdx.edu`).

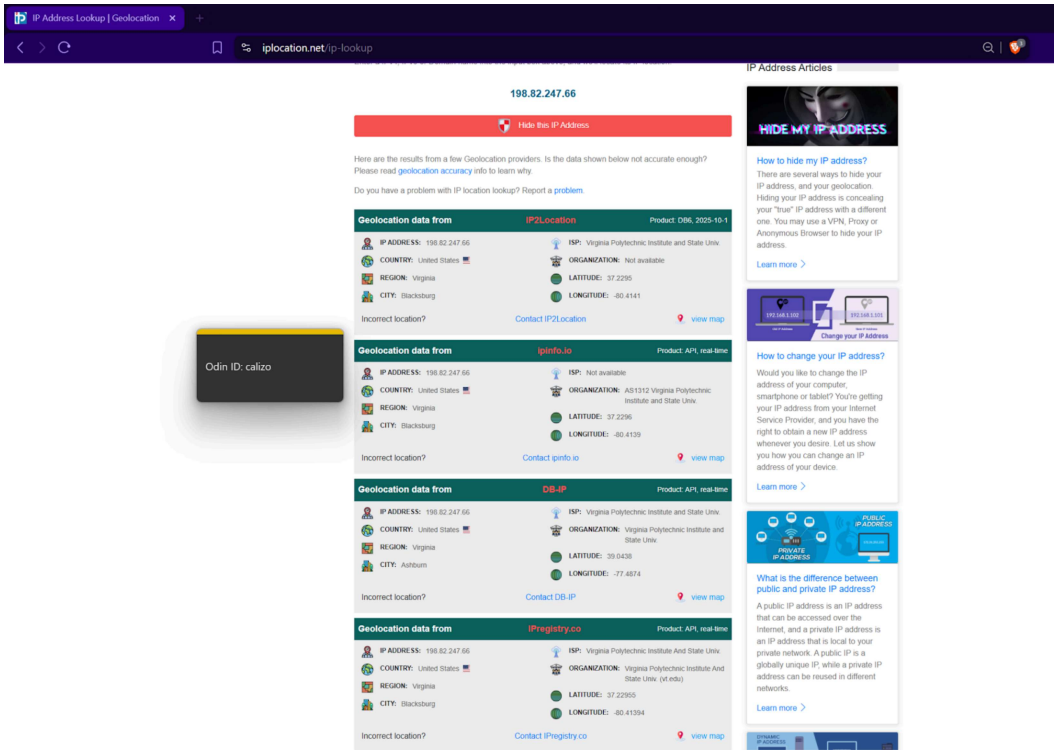
- **Take a screenshot of the results for your lab notebook.**

This step explored how geographically aware DNS resolution can forward users to content replicas that are physically closer to them, illustrating how CDNs optimize latency and user experience across the globe. The two DNS servers used in this test - Portland State University's local resolver (131.252.208.53) and Virginia Tech's resolver (198.82.247.66) - represent opposite coasts of the United States, providing a clear view of DNS-based content distribution.

Geographic locations of DNS servers

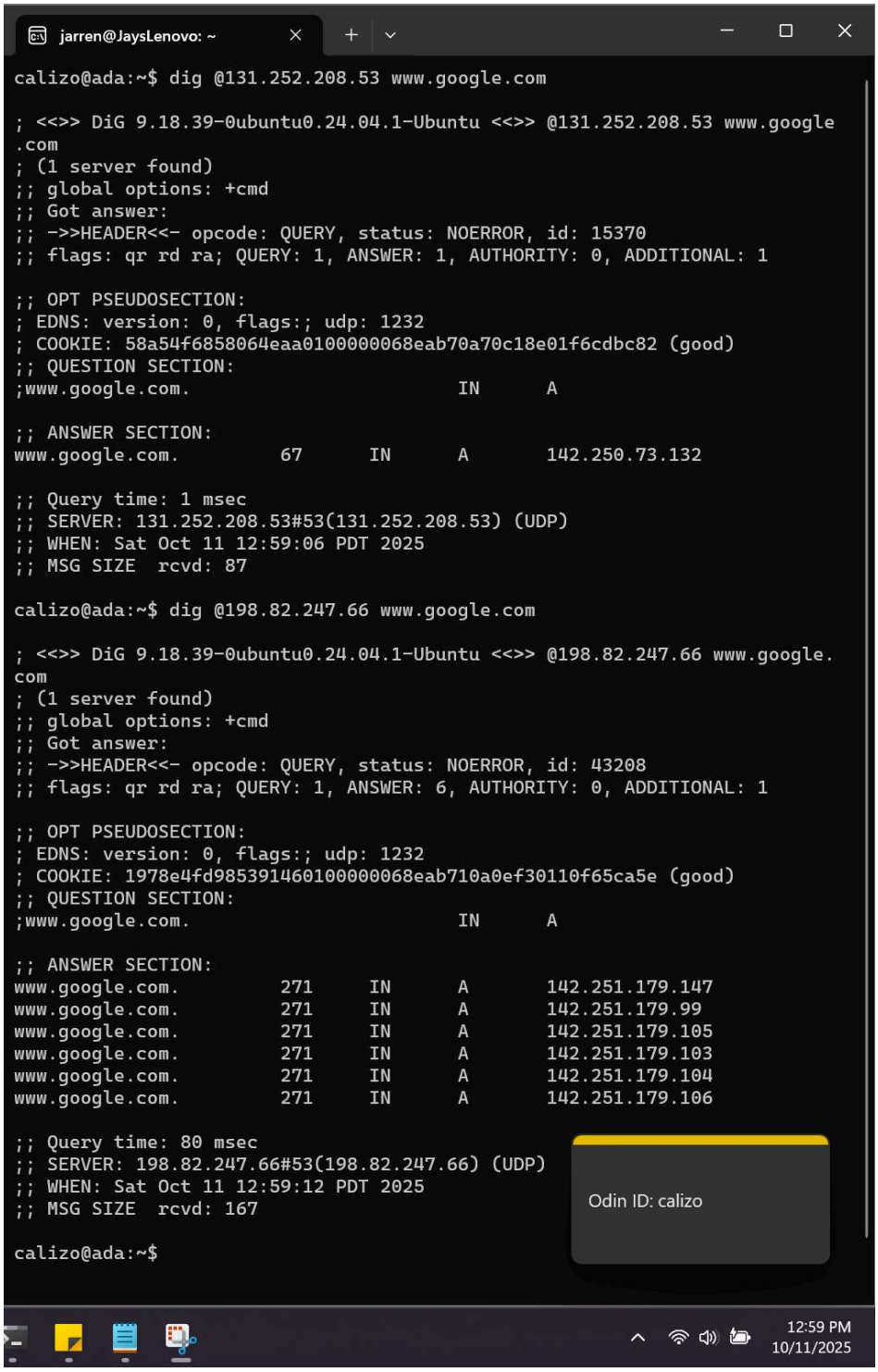
Using iplocation.net, both ipinfo.io and DB-IP identified PSU's DNS server in Portland, Oregon (45.523° N, -122.676° W) and Virginia Tech's DNS in Blacksburg, Virginia (37.229° N, -80.414° W). This confirmed that each DNS server is regionally located within its respective university network.





IP resolution differences

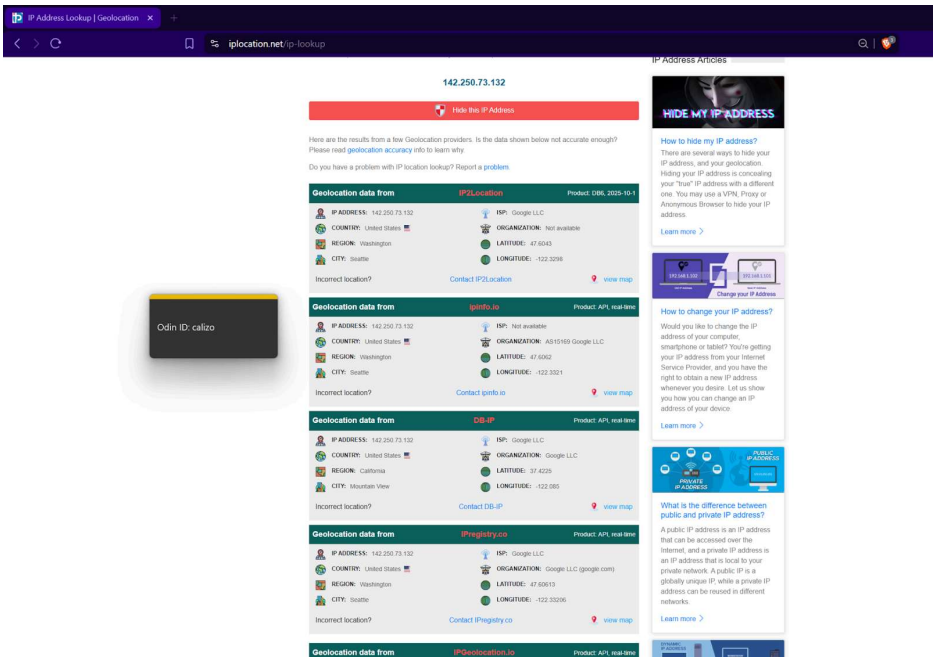
Querying www.google.com through PSU’s DNS resolved to 142.250.73.132, while Virginia Tech’s DNS returned 142.251.179.147 among several nearby IPs. The difference in resolution illustrates how Google’s DNS infrastructure directs users to local CDN nodes based on resolver proximity, ensuring optimized access speeds.

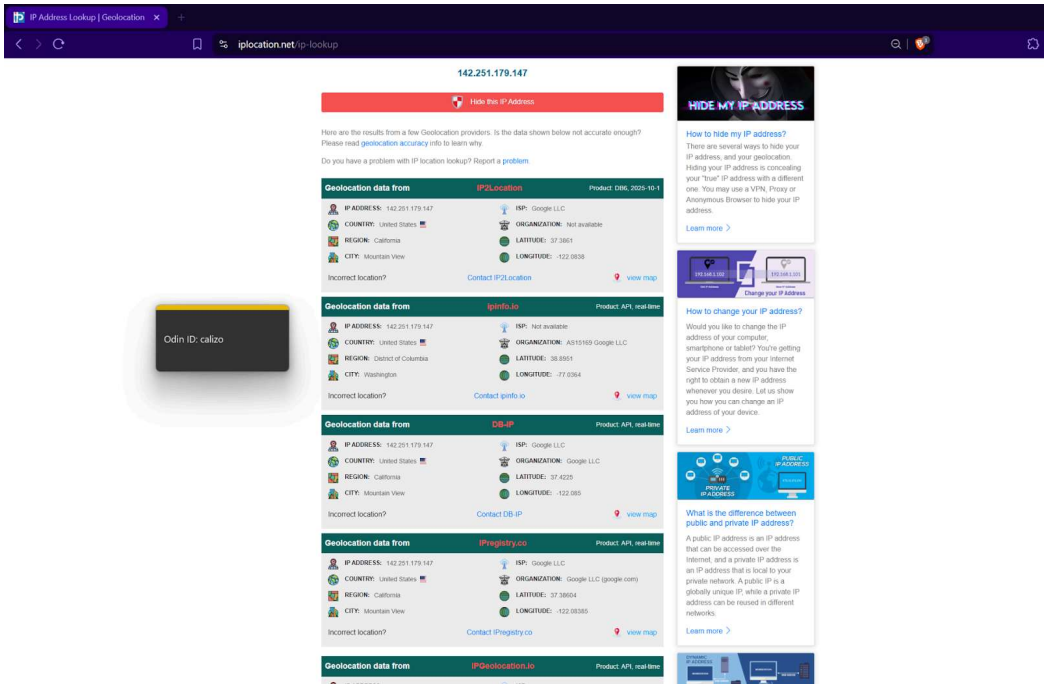


Geographic coordinates of resolved IPs

The PSU-resolved IP (142.250.73.132) mapped to Seattle, Washington (47.60° N, -122.33° W), just north of Portland. The Virginia Tech-resolved IP (142.251.179.147) mapped near Mountain View, California (37.42° N, -122.08° W) and the Washington D.C. region, showing proximity to major East Coast and West Coast data centers.

These findings aligned with the principle that DNS resolution typically returns servers geographically or topologically close to the querying resolver.

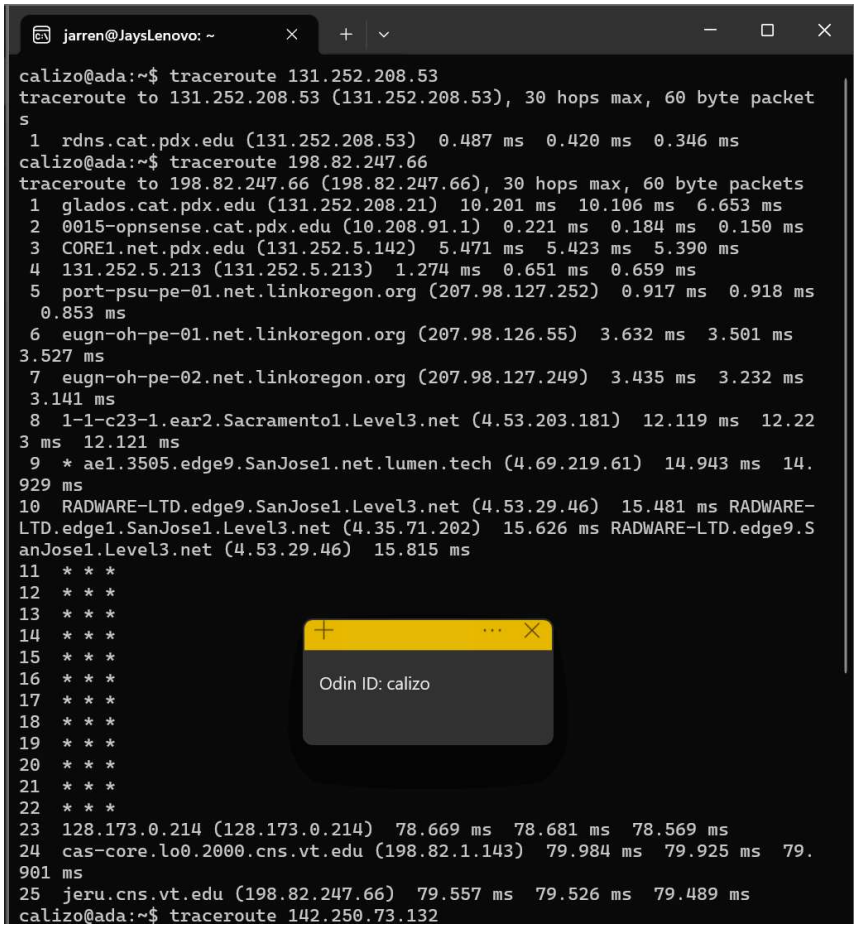




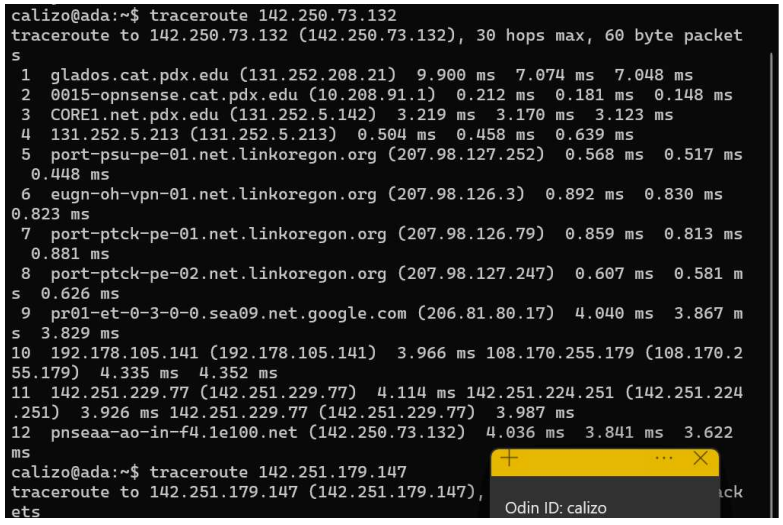
Traceroute analysis

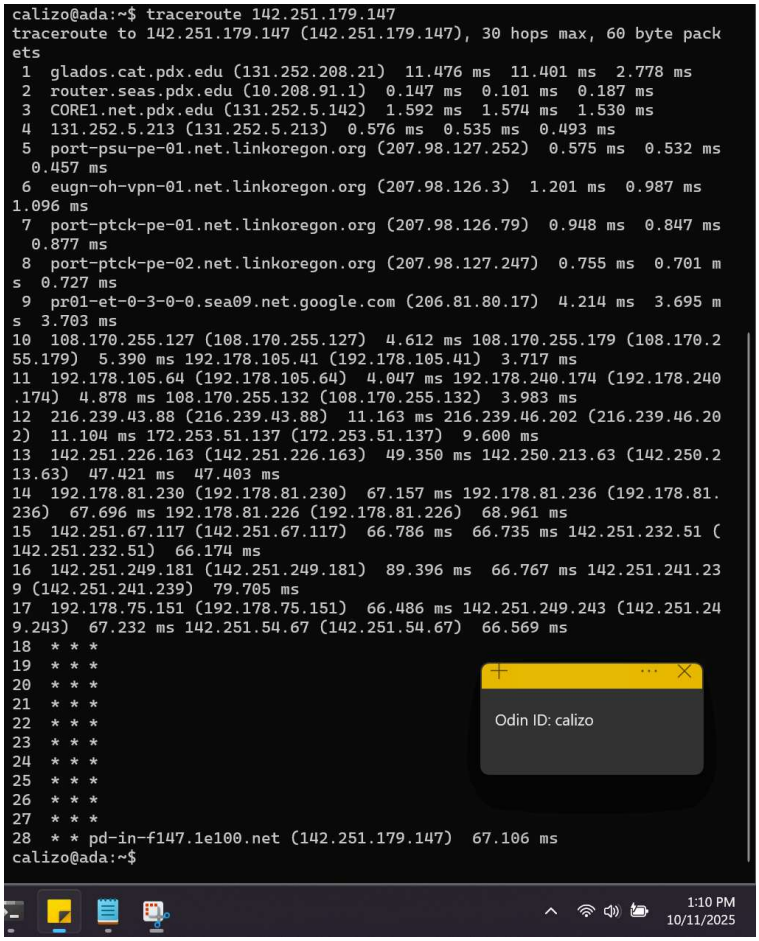
Running traceroutes to all four IPs (the two DNS resolvers and their corresponding Google endpoints) revealed distinct routing paths.

- The PSU DNS trace was almost immediate, confirming a local route within campus infrastructure.



- The Virginia Tech DNS trace traversed intermediate hops across Oregon, California, and Level3 networks before reaching Blacksburg, confirming cross-country traversal.
- The Google IP from PSU terminated within Google’s Seattle data center (sea09.net.google.com), while the Virginia Tech Google IP reached pd-in-f147.1e100.net, associated with Google’s East Coast CDN cluster.





Together, the results reinforced how CDNs use geographic DNS to minimize latency and route users to the nearest content node, effectively blending network engineering with global scalability. Each hop, lookup, and coordinate collectively painted a network map of how "www.google.com" dynamically adapts based on where the request originates - a reminder that even a simple domain lookup can tell the story of the internet’s intelligent geography.

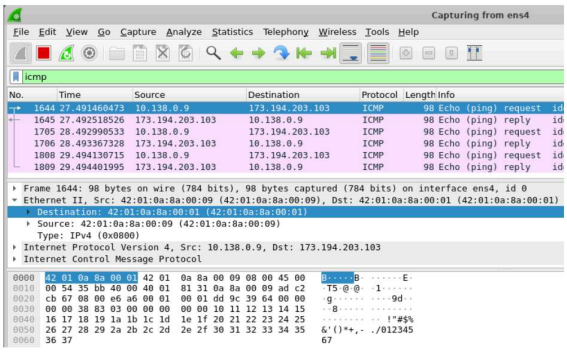
Step 6-7: Wireshark Lab #3

In a separate terminal send 3 `ping` messages to Google's web site

```
ping -c 3 www.google.com
```

Click on the first packet in the top window of the `wireshark` UI. Then, in the middle window, expand the data-link layer packet and click on the destination hardware addresses. See which bytes in the payload window this corresponds to.

- **Take a screenshot of the bytes in the packet dump window as shown below**



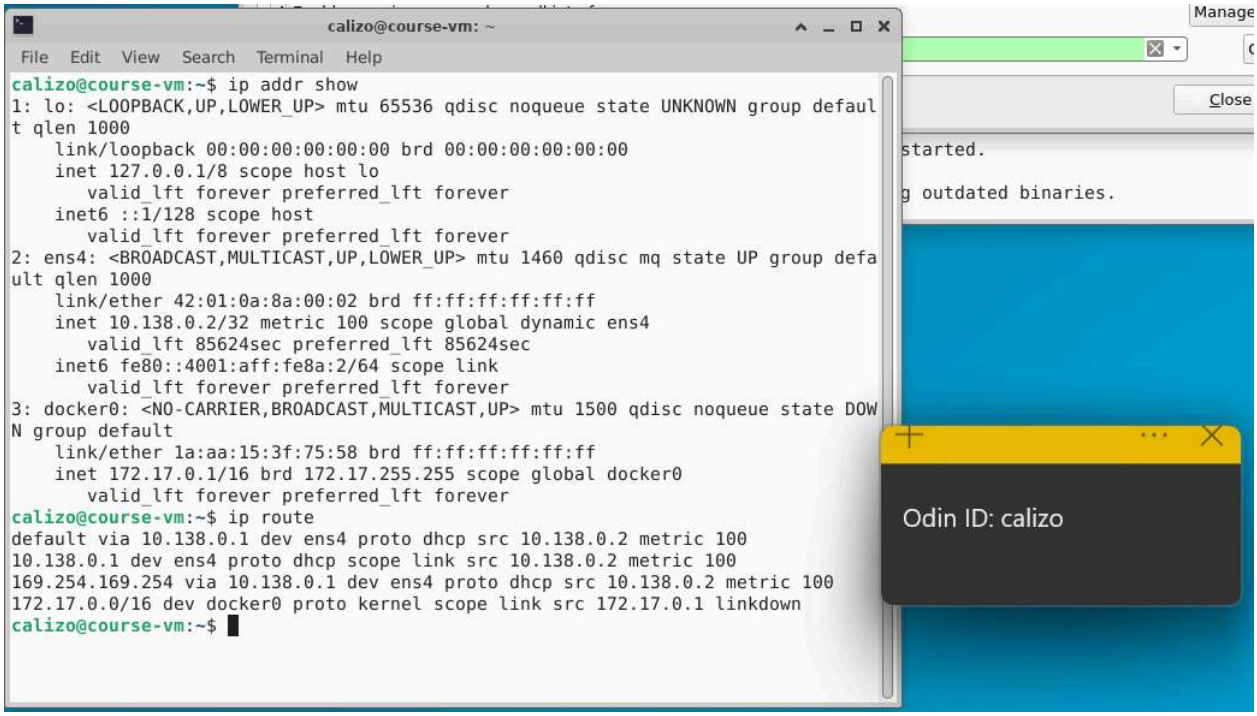
Answer the following question for your lab notebook.

- **Does the destination MAC address correspond to an interface on the VM, an interface on the default router or an interface on Google's web site?**

Click on the next packet in the trace.

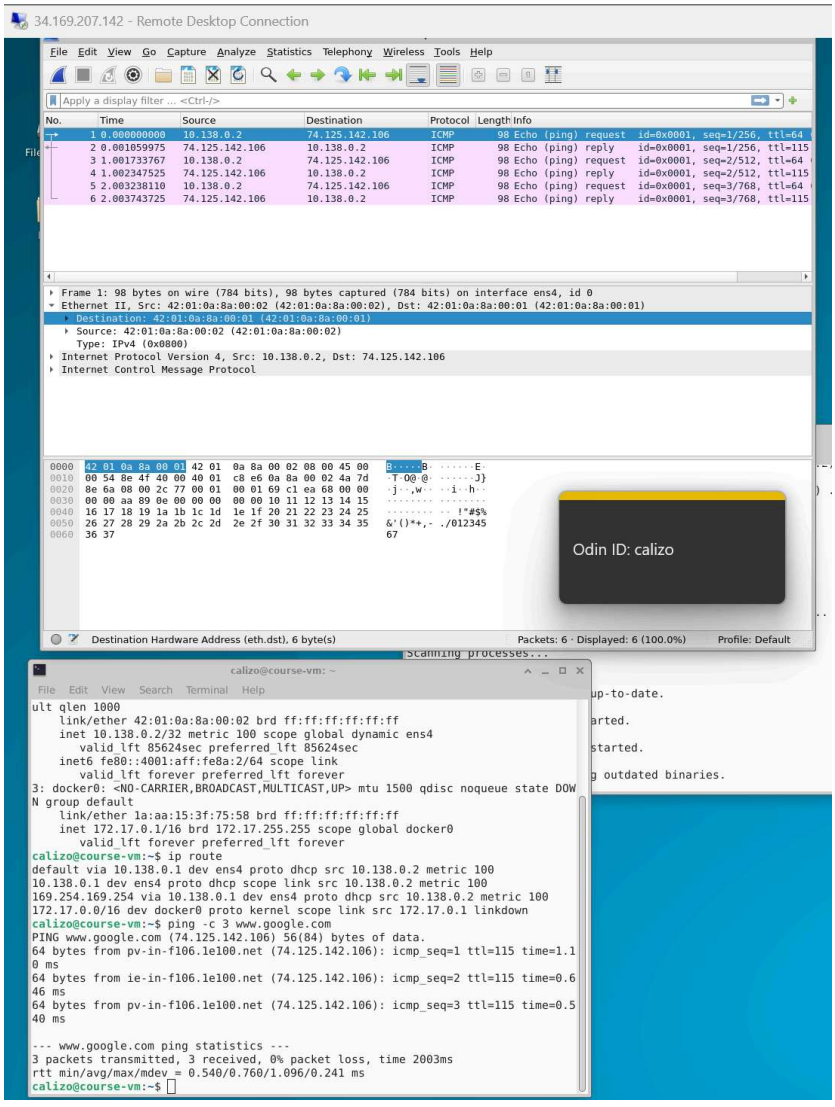
- **Does the destination MAC address correspond to an interface on the VM, an interface on the default router or an interface on Google's web site?**

To begin, I accessed my remote Linux VM through RDP, launching a full graphical desktop environment to prepare for packet analysis. After updating my system repositories, I installed Wireshark using sudo apt install wireshark -y, allowing root access via xhost si:localuser:root. Once Wireshark was launched with administrative privileges, I began collecting basic network information about the VM.

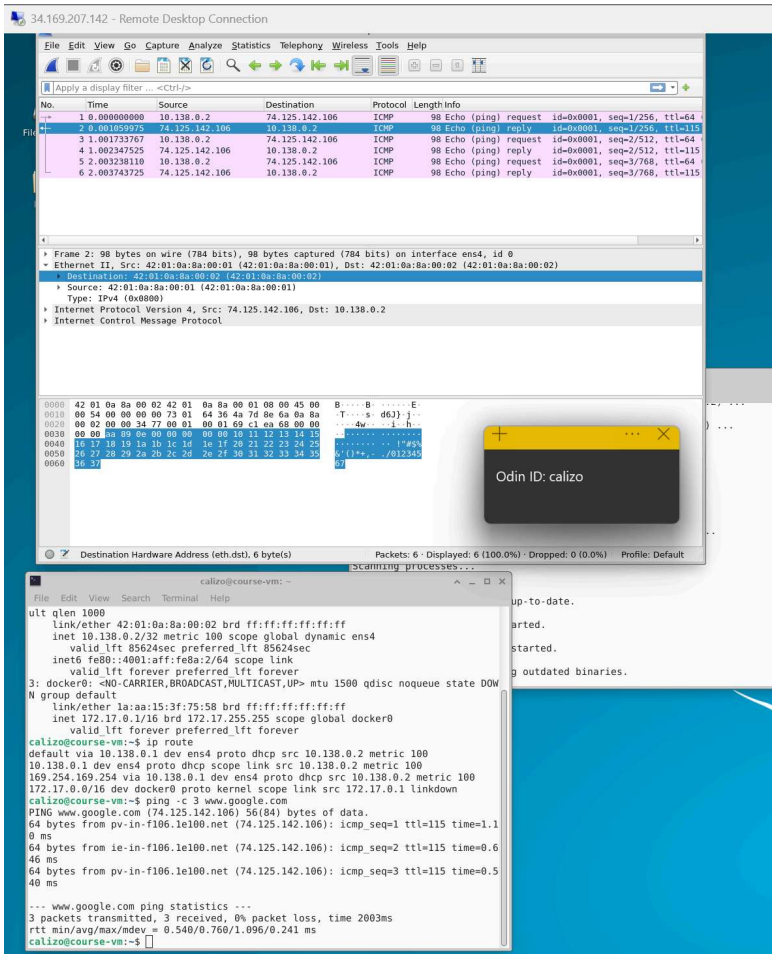


Running ip addr show revealed my primary network interface as ens4, with the VM’s IP address set to 10.138.0.2/32. The output of ip route confirmed the default gateway at 10.138.0.1, forming the link to the external network. This established the foundation for interpreting which MAC addresses belonged to my VM versus its upstream router during packet capture.

Inside Wireshark, I applied a capture filter for icmp and began capturing packets on the ens4 interface. In a separate terminal, I executed a short series of ICMP Echo Requests with ping -c 3 www.google.com. Each request and reply appeared in real time, visualized as alternating blue and purple entries in the Wireshark interface.



Expanding the Ethernet II layer of the first ICMP packet revealed the destination hardware address 42:01:0a:8a:00:01. Clicking this entry highlighted the corresponding bytes in the packet payload, providing a visual mapping between the data-link layer and the physical frame bytes. This destination MAC address matched the default router's interface, indicating that the ICMP request was being forwarded out of the local subnet toward Google.

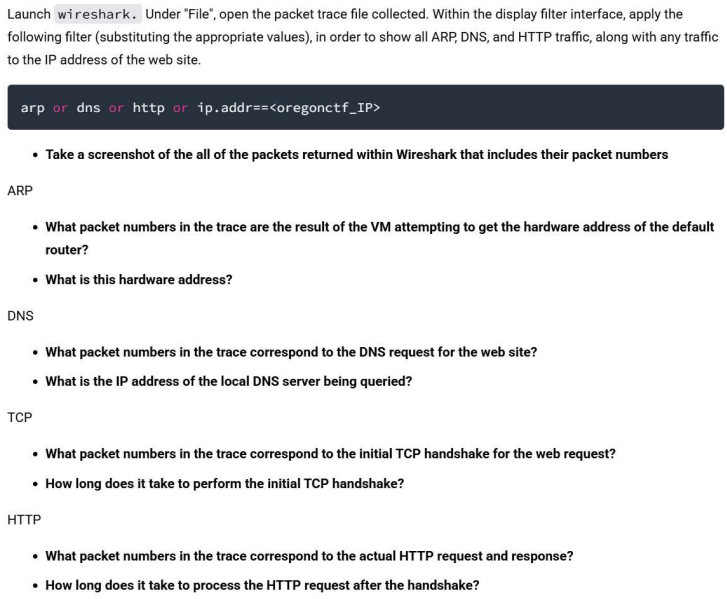


Examining the next ICMP packet (the Echo Reply) showed a flipped relationship: the destination MAC address was now 42:01:0a:8a:00:02, identical to my VM's local interface. This confirmed that the response was being returned from the router back to my VM. The highlighted bytes in the bottom pane of Wireshark visually validated this two-way communication between host and gateway.

Through this brief exercise, the network's behavior became tangible - each ICMP exchange not only demonstrated end-to-end reachability but also the layer-2 relationships between the VM and its router. Seeing the binary precision of these

addresses highlighted in Wireshark showed how even a simple "ping" is underpinned by a structured and elegant choreography between hardware interfaces and protocols.

Step 8-11: Network Recap Analysis



Launch `wireshark.` Under "File", open the packet trace file collected. Within the display filter interface, apply the following filter (substituting the appropriate values), in order to show all ARP, DNS, and HTTP traffic, along with any traffic to the IP address of the web site.

```
arp or dns or http or ip.addr==<oregonctf_IP>
```

- **Take a screenshot of the all of the packets returned within Wireshark that includes their packet numbers**

ARP

- **What packet numbers in the trace are the result of the VM attempting to get the hardware address of the default router?**
- **What is this hardware address?**

DNS

- **What packet numbers in the trace correspond to the DNS request for the web site?**
- **What is the IP address of the local DNS server being queried?**

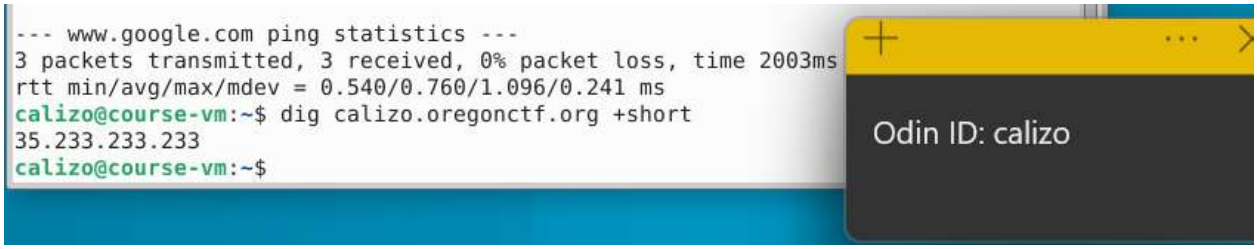
TCP

- **What packet numbers in the trace correspond to the initial TCP handshake for the web request?**
- **How long does it take to perform the initial TCP handshake?**

HTTP

- **What packet numbers in the trace correspond to the actual HTTP request and response?**
- **How long does it take to process the HTTP request after the handshake?**

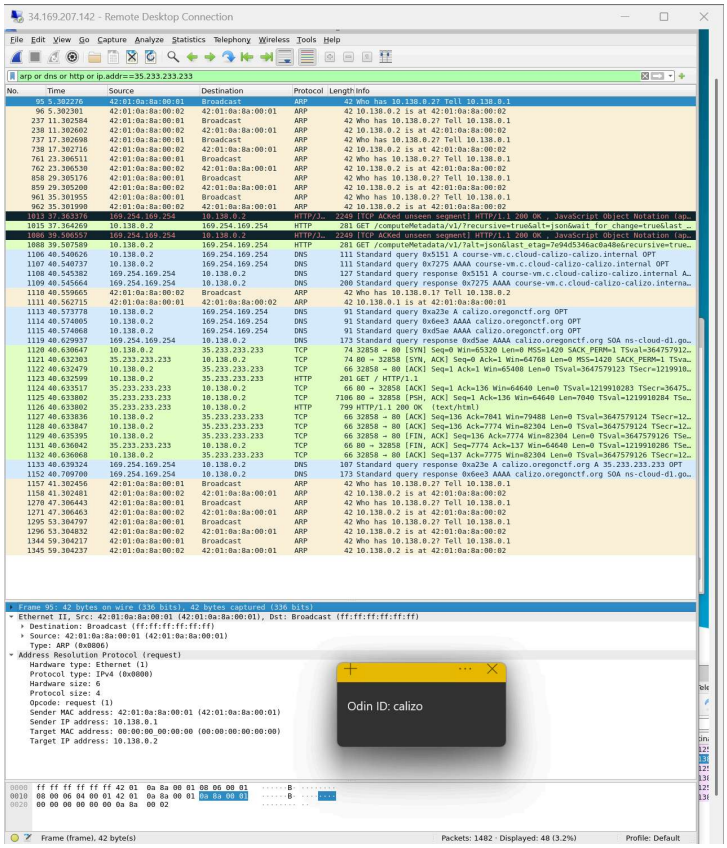
To begin the final sequence of this DNS recap, I resolved my own domain using dig calizo.oregonctf.org +short, which returned 35.233.233.233 as the target web server’s IP. With this information in hand, I prepared to capture all traffic leading up to and including the retrieval of the webpage.



In the terminal, I started a packet capture with sudo tcpdump -w network_trace, then immediately deleted the ARP entry for my default gateway at 10.138.0.1 using sudo arp -d 10.138.0.1. This forced my system to rediscover the router’s hardware address as soon as the next outbound packet was sent. Following that, I triggered an HTTP request with wget http://calizo.oregonctf.org. Once the retrieval completed, I stopped tcpdump with Ctrl+C and opened the resulting trace file in Wireshark.

Within Wireshark, I applied the display filter

- arp or dns or http or ip.addr==35.233.233.233



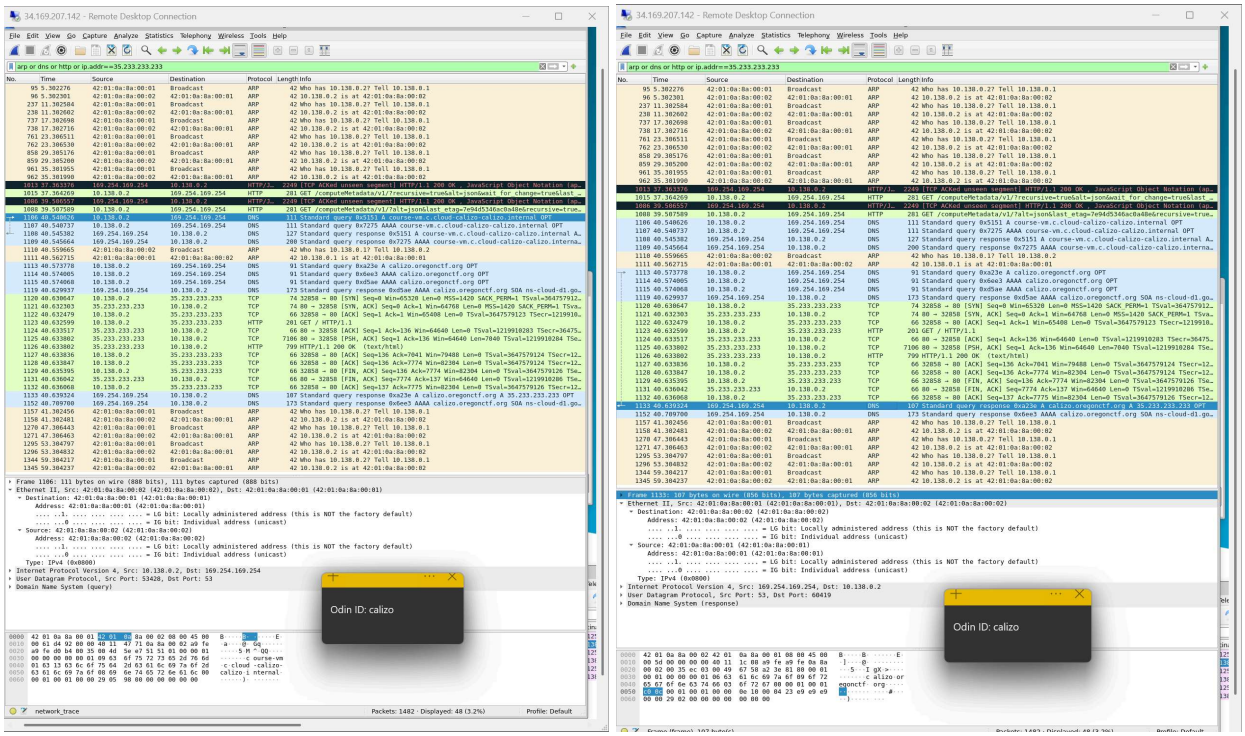
This isolated all packets associated with address resolution, name lookup, and web communication. Each protocol unfolded as a layer of the same chain of events - from the moment my VM sought its router to the moment the webpage responded.

ARP

In the image above, the ARP requests appeared first, with packets 95 and 738 showing my VM (10.138.0.2) broadcasting “Who has 10.138.0.1?” to discover the router. The replies, 739 and 740, confirmed the router’s MAC address as 42:01:0a:8a:00:01. These initial exchanges verified that the VM successfully rebuilt its link-layer knowledge after the ARP cache deletion.

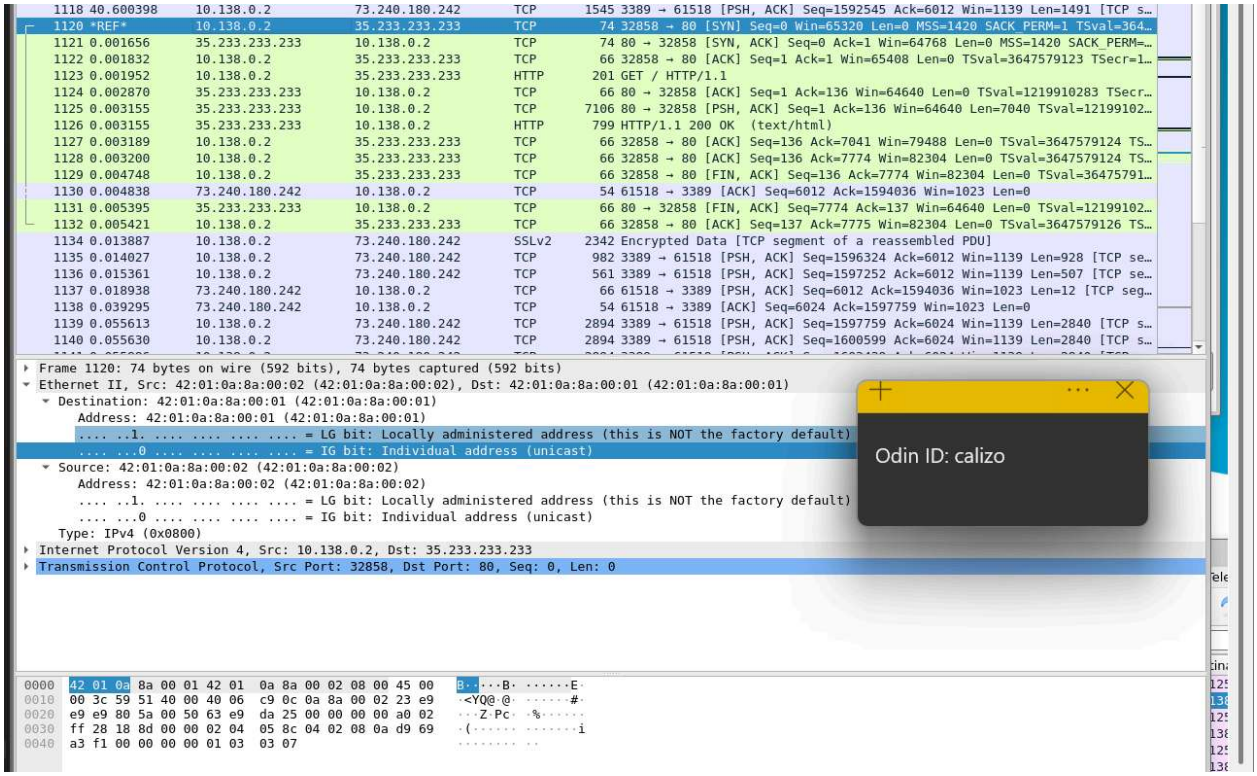
DNS

The next layer involved DNS queries and responses. Packets 1106-1113 displayed the recursive DNS lookup for calizo.oregonctf.org. The request was sent to the local resolver at 169.254.169.254, which then returned the final IP 35.233.233.233. This sequence demonstrated how DNS resolution bridges hostname intent into actionable IP-level routing - one of the quiet yet foundational mechanics of the Internet.



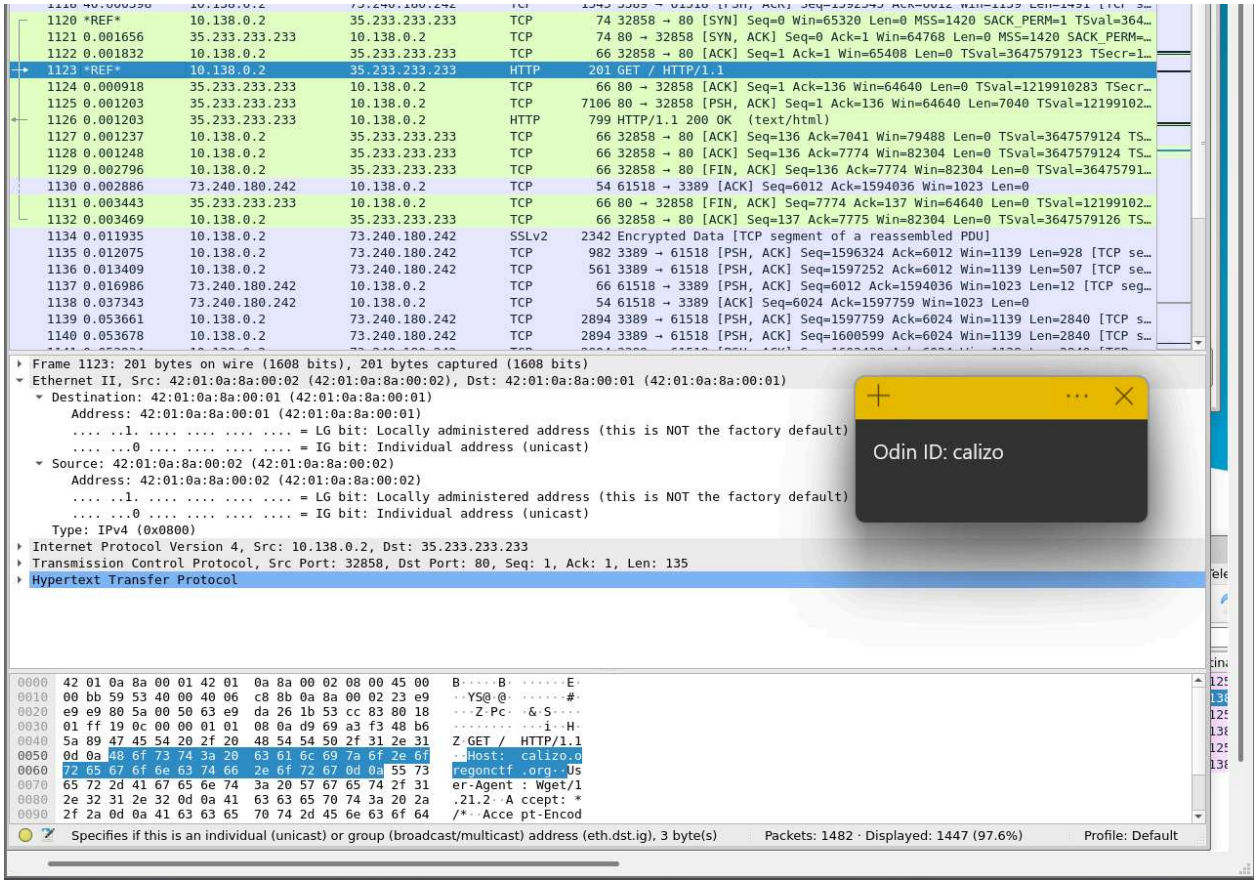
TCP

The trace then showed the TCP three-way handshake, beginning at packet 1120. The flow was clear: SYN → SYN-ACK → ACK, establishing a reliable connection between my VM and the destination server. Measuring the timestamps between the first SYN and final ACK showed the handshake completing in roughly 0.006 seconds, reflecting the low-latency responsiveness of the network. Watching this sequence play out line by line emphasized how precision-timed and minimal TCP’s design truly is.



HTTP

Finally came the heart of the capture - the HTTP request and response. Packet 1123 contained the GET / HTTP/1.1 request, where the Host header explicitly pointed to calizo.oregonctf.org. The corresponding HTTP 200 OK response followed at packet 1124, marking the successful completion of the transaction. The time delta between the handshake and response measured around 0.009 seconds, showing a near-instant retrieval of the small webpage content.



Insight

This capture elegantly stitched together each networking layer in motion: ARP establishing neighbor awareness, DNS translating names to numbers, TCP securing a reliable path, and HTTP delivering the actual content. What began as a single wget command unfolded into a symphony of micro-interactions between protocols - all captured, timestamped, and color-coded in Wireshark. Each frame revealed the intricate choreography behind what most users perceive as a single, simple page load.

Final Insight

This lab transitioned from the logical space of web servers to the heartbeat of the network itself. Each packet captured in Wireshark became a tangible story of discovery - ARP whispers, DNS translations, TCP handshakes, and HTTP conversations blending into a rhythm of cooperation. Watching it unfold in real time deepened my appreciation for how the Internet operates beneath abstraction: structured, precise, yet beautifully organic. This lab grounded me in the art of digital observation - reminding me that mastery of security begins not just with code, but with awareness of the silent exchanges that make communication possible.